KOSTA AI LAB · AI NATIVE SDLC

AI-DLC 스펙주도개발

의도를 먼저 고정하고, 코드를 생성합니다

AI-DLC 방법론과 두 구현체 OpenSpec · Spec Kit을 같은 예제로 실습하고 비교합니다

방법론 | OpenSpec | Spec Kit | 거버넌스

OVERVIEW · 과정 소개

과정 개요

AI-DLC를 상위 방법론으로, OpenSpec·Spec Kit을 그 구현체로 다룹니다

도구 이전에 방법론

방법론	AI-DLC(생명주기) · SDD(명세 원칙)
도구	OpenSpec · Spec Kit — 같은 방법론의 두 구현체
에이전트	Claude Code · Cursor — 실제로 코드를 쓰는 실행기

살아있는 교재로 실습합니다

같은 상품·주문 e-커머스 API를 두 도구로 각각 개발한 **재현 가능한 git 스냅샷**을 실습 자산으로 씁니다. 어느 단계든 `git checkout`으로 되살려 따라갑니다.

OVERVIEW · 과정 소개

오늘의 시간표

1일 8블록으로 방법론(LO1~4)부터 두 도구 실습(LO5~8)과 비교·도입(LO9~10)까지 진행합니다 (09:00–17:50, 점심 12:00–13:00)

시간	블록	내용
09:00–09:50	1 · 오프닝	과정 소개 + LO1 방법론 스펙트럼(바이브→AI-DLC→SDD)
10:00–10:50	2 · 이론	LO2 AI-DLC 생명주기와 인간·AI 역할
11:00–11:50	3 · 이론	LO3 SDD 원리 + LO4 산출물 지도
12:00–13:00	점심	
13:00–13:50	4 · 실습	LO5 OpenSpec 기초 — 상품 CRUD 관통 실습 착수
14:00–14:50	5 · 실습	LO6 OpenSpec 심화 — delta 변경(주문·재고)
15:00–15:50	6 · 실습	LO7 Spec Kit 기초 — 같은 CRUD를 Spec Kit으로
16:00–16:50	7 · 실습	LO8 Spec Kit 심화 + LO9 정면 비교
17:00–17:50	8 · 판단	LO10 조직 도입 로드맵 + 클로징·회고

OVERVIEW · 과정 소개

학습 목표

이 과정을 마치면 방법론의 원리부터 두 도구의 실전 사용, 조직 도입 판단까지 이어서 설명하고 수행할 수 있습니다

1. **AI-DLC 생명주기와 역할 분담**
   AI가 실행하고 사람이 감독하는 생명주기, 인간·AI의 역할 경계를 설명합니다.
2. **SDD 원리를 산출물에 적용**
   의도가 진실의 원천이라는 원리와 ACME를 proposal·spec 산출물에 적용합니다.
3. **OpenSpec 스펙→코드 사이클**
   artifact-guided 사이클과 delta 변경 관리(MODIFIED)를 직접 수행합니다.
4. **Spec Kit 워크플로우**
   constitution → specify → plan → tasks → implement를 같은 예제로 수행합니다.
5. **비교와 조직 도입 판단**
   두 구현체를 비교해 프로젝트에 맞는 선택과 도입·거버넌스 계획을 세웁니다.

OVERVIEW · 과정 소개

강사 소개: 윤성열

(주)프렌티스 대표이사 · 공학박사

17년 이상	8년 이상	특허 4건	공학박사
SW 아키텍처	기업 교육/컨설팅	AI/에이전트 시스템	가천대 전자계산학

주요 경력

2025~현재	대표이사	(주)프렌티스 — AI 플랫폼 기술개발 총괄, Spectra/CodeRay 제품화
2023~현재	실장	(주)닥터소프트 — SaaS Management Platform 개발 총괄
2018~현재	전문강사	KOSTA — AI/클라우드/SW개발 기업 교육, 과정기획 자문
2020~2022	연구소장	(주)네비웍스 — 메타버스 서비스 개발 총괄
2017~2018	겸임교수	가천대학교 — 대학원 융합서비스/모바일프로그래밍

교육/컨설팅 고객

KT / KTDS	AI 기반 SW 개발, Agentic AI
SK플래닛	AI 기반 SW 개발 방법론
글로벌넬리지(AIVLE)	AI, 컨테이너, Agentic AI
KLCNS / 페타시스	Private AI 도입 컨설팅
경남대 / 가천대	디지털 기획, SW 교육

SECTION

01 AI 시대 개발 방법론 스펙트럼

- 세 방식(바이브·AI-DLC·SDD)을 하나의 스펙트럼으로 배치
- 바이브 코딩의 한계를 구체적 데이터로 제시
- 왜 지금 스펙주도개발인가를 프로젝트 상황에 대입해 판단
- 방법론(AI-DLC·SDD)과 도구(OpenSpec·Spec Kit)의 관계 구분

50분

개념

개발 병목의 이동

코드는 AI가 짜고, 사람은 무엇을 만들지 정합니다

90%
Anthropic 사내 코드 중 AI가 작성하는 비중 (Dreamforce 2025)

84%
AI 도구를 쓰거나 도입 계획인 개발자 (Stack Overflow 2025)

51%
AI 도구를 매일 쓰는 전문 개발자

코드를 짜는 손은 더 이상 병목이 아닙니다. 개발자의 역할은 코드를 직접 치는 **코더**에서, AI에게 방향을 주고 결과를 검증하는 **오케스트레이터**로 옮겨갑니다. 지휘자가 모든 악기를 연주하지 않아도 전체 음악을 이끄는 것과 같습니다.

Anthropic Dreamforce 2025 · Stack Overflow Developer Survey 2025 · Dario Amodei(2025.3)

설명

개발 방법론 스펙트럼

자유도와 예측 가능성을 맞바꾸는 하나의 축입니다

구분	바이브 코딩	AI-DLC	스펙주도개발(SDD)
한 줄 정의	느낌으로 지시하고 결과만 본다	AI가 실행하고 팀이 감독하는 생명주기	스펙을 먼저 고정하고 코드를 생성
진실의 원천	채팅 히스토리	요구사항 + 스펙	의도(스펙)
인간 역할	방향 제시 + 수용	검증자 + 의사결정자	스펙 저자 + 검증자
자유도 / 예측	높음 / 낮음	중간 / 중간	낮음 / 높음
적합 범위	프로토타입·탐색	팀 단위 엔터프라이즈	프로덕션·유지보수
대표 도구	Cursor, Claude Code	Amazon Q, Kiro	OpenSpec, Spec Kit

개념

세 방법론의 정의

바이브 코딩·AI-DLC·SDD는 등장 배경도 시점도 다릅니다

Karpathy, 2025.2

바이브 코딩

보고·말하고·실행·복붙하면 대체로 작동한다는 방식. 코드를 자세히 읽지 않고 실행 결과만으로 판단합니다. 개발의 민주화를 앞당겼으나 프로덕션에는 취약합니다.

예시: 자유도 최상 · 프로토타입·탐색에 최적

AWS DevSphere, 2025.8

AI-DLC

AI가 실행하고 사람이 감독한다를 원칙으로, 바이브 코딩의 에너지를 팀과 엔터프라이즈 수준으로 끌어올린 생명주기입니다.

예시: 협업 구조 · 팀 단위 엔터프라이즈

명세 우선 철학

스펙주도개발 (SDD)

코드보다 명세가 먼저라는 철학. 의도(스펙)가 진실의 원천이며 코드는 그 스펙을 특정 언어로 표현한 결과입니다. 버그란 스펙을 잘못 구현한 것입니다.

예시: 예측 가능성 · 프로덕션·유지보수

설명

방법론과 도구의 구분

도구는 바뀌어도, 방법론은 그대로 적용됩니다

방법론	AI-DLC(큰 생명주기) · SDD(그 안의 명세 원칙) — 어떻게 일할 것인가
도구	OpenSpec(브라운필드) · Spec Kit(그린필드) — 방법론을 실천하는 구현체
에이전트	Claude Code(Opus 4.8) · Cursor · OpenCode — 실제로 코드를 쓰는 실행기

이 과정이 도구 이전에 방법론을 먼저 세우는 이유입니다. 두 도구는 같은 방법론의 두 구현체입니다.

심화

바이브 코딩의 한계

프로토타입에는 훌륭하지만, 프로덕션에서는 한계가 드러납니다

🚫 ERROR 한계 1 · 보안 취약점

코드는 3~4배 많이 생성했지만 보안 이슈는 10배 늘었습니다(Fortune 50 분석). AI 코드 최대 40%가 SQL 인젝션에 취약합니다.

Lovable 웹앱 1,645개 중 170개에서 개인정보가 누구나 접근 가능

⚠️ WARNING 한계 2 · QA 생략

실천자의 36%가 품질 보증을 완전히 건너뜁니다. 에러 없이 실행되면 품질이 괜찮다고 간주합니다.

개발자 68%가 AI 코드를 빠르지만 결함 있음으로 인식

⚠️ WARNING 한계 3 · 레거시·기술 부채

AI 생성 코드는 사람 코드보다 수정률(churn)이 41% 높고, 코드 복제(clone)는 4배 늘었습니다.

GitClear 2024, 2억 1,100만 라인 분석

ℹ️ INFO 한계 4 · 컨텍스트 소실

채팅으로 이어가면 요구사항이 뒤섞이고, 대화를 요약(compact)하면 왜 그렇게 결정했는가가 함께 날아갑니다.

코드에는 의도가 없습니다 — 코드는 의도의 결과일 뿐입니다

예제

바이브 한계 수치

네 한계의 공통 뿌리는, 의도를 붙잡아 둘 곳이 없다는 것입니다

한계 지표	수치	출처
보안 이슈	코드량 3~4배 대비 10배 (Fortune 50)	arxiv 2510.00328
SQL 인젝션 취약	AI 코드 최대 40%	ZenCoder
QA 생략	실천자 36%	학술 리뷰
코드 수정률(churn)	사람 대비 41% 높음	GitClear 2024
코드 복제(clone)	4배 증가	GitClear 2024
프로젝트 포기	바이브 세션의 11%	학술 리뷰

설명

의도가 빠진 코드

코드는 무엇을 하는지 보여줄 뿐, 왜 그렇게 했는지는 남기지 않습니다

처음 요구사항
채팅 첫머리에 설명하지만 대화가 길어지며 묻힙니다

중간 변경
요구가 바뀌어도 어디서 왜 바뀌었는지 남지 않습니다

요약(compact)
왜 이렇게 결정했는가 같은 근거가 함께 날아갑니다

리버스 엔지니어링
완성된 코드·DB로는 의도를 되살릴 수 없습니다

코드는 의도의 결과일 뿐입니다.

스펙주도개발은 이 의도를 문서로 잠가(lock) 진실의 원천으로 삼고, 결과물을 그 스펙과 대조해 검증합니다. 바이브 코딩이 문제가 아니라, 프로덕션으로 갈 때 다음 단계가 필요할 뿐입니다.

예제

두 방식의 결과 비교

같은 기능도 바이브는 빈칸을 AI가 채우고, 스펙주도는 검증 기준을 남깁니다

관점	바이브 코딩	스펙주도개발
지시	할 일 삭제 기능 만들어줘	ACME로 가정·제약·지표·예시를 명시한 스펙 작성 후 생성
권한 검사	AI가 임의 판단(대개 누락)	본인 Todo만 삭제, 타인 요청은 403 명시
없는 ID 삭제	정의 안 됨(에러? 무시?)	이미 삭제된 항목 재삭제 시 404 명시
성능 기준	빠르게(측정 불가)	응답 p95 200ms 이하(측정 가능)
검증	실행되면 통과로 간주	스펙의 예시·시나리오와 대조

바이브는 5분이면 동작하는 코드를 얻지만 빈칸을 AI가 제멋대로 채웁니다. 스펙주도는 시간을 더 쓰는 대신 객관적으로 검증할 기준을 함께 얻습니다.

개념

스펙트럼상의 선택

우열이 아니라, 상황에 따라 고르는 선택입니다

주말 프로토타입, 빠른 아이디어 검증

바이브 코딩

팀이 함께 만드는 엔터프라이즈 기능

AI-DLC

오래 유지보수할 프로덕션 코드

스펙주도개발

> **AI가 90%를 짜더라도 무엇을 만들지는 사람이 정의합니다. 우리는 여전히 개발을 하는 것이지, 단지 코딩을 대신 시키는 것이 아닙니다.**

레퍼런스

이 LO의 주장은 모두 1차·공개 출처에 근거합니다

코드 90%

Anthropic · Dario Amodei

Dreamforce 2025에서 사내 코드 90%가 AI로 작성됨을 공개. Amodei는 2025.3에 3~6개월 내 코드 90%를 예측했습니다.

예시: anthropic.com

사용률

Stack Overflow Developer Survey 2025

AI 도구 사용·도입 계획 84%, 전문 개발자 매일 사용 51%, AI 코드를 빠르지만 결함 있음으로 인식.

예시: survey.stackoverflow.co

코드 품질

GitClear · AI Copilot Code Quality 2024

2억 1,100만 라인 분석. AI 코드 수정률(churn) 41% 증가, 코드 복제(clone) 4배.

예시: gitclear.com

한계 데이터

Vibe Coding 학술 리뷰

QA 생략 36%, 프로젝트 포기 11%, 보안 이슈 10배 등 프로덕션 한계를 종합.

예시: arxiv.org/html/2510.00328v1

방법론 원류

AWS DevSphere 2025 · Andrej Karpathy

AWS는 AI-DLC(AI가 실행·사람이 감독)를 발표. Karpathy는 2025.2 바이브 코딩을 소개한 뒤 Nanochat을 수동 코딩하며 한계를 회고했습니다.

예시: aws.amazon.com · Karpathy 2025.2

체크포인트

핵심 점검

다음 다섯 가지를 자신의 언어로 설명할 수 있는지 확인해 봅니다

01 스펙트럼 배치

바이브·AI-DLC·SDD를 자유도와 예측 가능성 축에 놓고 각 적합 범위를 말할 수 있다

02 의도의 부재

코드에는 의도가 없다가 왜 컨텍스트 소실 문제와 연결되는지 설명할 수 있다

03 한계의 근거

바이브 코딩의 네 한계 중 둘을 구체적 수치와 함께 제시할 수 있다

04 방법론과 도구

AI-DLC·SDD(방법론)와 OpenSpec·Spec Kit(도구)을 세 층위로 구분할 수 있다

05 나의 선택

지금 프로젝트가 스펙트럼 어디에 서야 하는지 근거와 함께 판단할 수 있다

SECTION

02 AI-DLC 생명주기와 인간·AI 역할

- AI-DLC 생명주기(의도 → 스펙 → 계획 → 구현 → 검증)를 단계별로 설명합니다
- 각 단계에서 사람이 결정하는 것과 AI가 실행하는 것을 정확히 구분합니다
- AI의 자기 관대성과 멀티 에이전트 검증이 왜 품질을 높이는지 설명합니다
- 코더에서 오케스트레이터로 바뀌는 개발자 역할 변화를 자신의 직무에 대입합니다

50분 (블록 2)

개념

AI-DLC 개념

AI를 실행자로, 사람을 감독자로 두는 AWS의 개발 방법론입니다

AI가 실행하고, 사람이 감독한다

AWS AI-DLC 핵심 원칙

AI Powered Execution with Human Oversight — AI proposes, human approves

> AI는 계획을 세우고 코드를 짜고 테스트를 만드는 실행자입니다. 사람은 방향을 제시하고 결정을 내리고 결과를 검증하는 감독자입니다. AI는 반복 코드·패턴 인식·대량 문서 생성에 강하고, 사람은 비즈니스 맥락·창의적 문제 해결·윤리적 판단에서 우위입니다. 속도는 AI에서 얻고, 방향과 품질은 사람이 통제합니다.

AWS, 'AI-Driven Development Life Cycle: Reimagining Software Engineering' (AWS DevOps Blog); re:Invent 2025 DVT214

설명

애자일 용어 재정의

Sprint는 시간·일 단위 Bolt로, Epic은 Units of Work로 바뀝니다

기존 애자일	AI-DLC	의미
Sprint (2~4주)	Bolt (시간·일 단위)	반복 주기를 시간·일 단위로 압축해 피드백을 빠르게 돌립니다
Epic (추상적)	Units of Work	AI가 한 번에 처리하는 명확한 작업 단위로 바꿔 실행 가능하게 합니다

8~12주 → 2~3주

프로젝트 기간 단축

AWS 발표

모듈 4개 · 20시간

프로덕션 준비 모듈 구축

Wipro CTO 발표

AWS AI-DLC (re:Invent 2025 DVT214), AWS DevSphere 2025; Wipro CTO Sandhya Arun 발표

설명

생명주기 다섯 단계

AWS의 세 단계를 스펙주도 관점에서 더 잘게 나눈 흐름입니다



검증 ↺ 스펙 복귀 한 번 흐르고 끝나는 폭포수가 아닙니다. 검증에서 문제가 나오면 코드를 땜질하지 않고 **스펙으로 되돌아가고**, 운영 중 나온 피드백이 다시 의도로 이어지는 **순환 루프**입니다. 스펙은 프로젝트와 함께 진화하는 살아있는 문서입니다.

AWS, 'AI-Driven Development Life Cycle' (AWS DevOps Blog); re:Invent 2025 DVT214 — 3단계(Inception·Construction·Operation) 기준

설명

단계별 활동

의도는 사람이, 스펙과 구현은 Mob으로 함께 만들고, 검증은 스펙과 대조합니다

1 **의도(Intent)**
무엇을 왜 만들지 비즈니스 의도를 정합니다. AI가 만들어 줄 수 없는, 온전히 사람의 몫입니다.
사람 정의

2 **스펙(Spec)**
AI가 초안과 빈틈 질문(에러 알림·동시 접속 제한 등)을 던지고, 사람이 실시간으로 답하며 ACME로 채웁니다. 산출물은 Units of Work입니다.
Mob Elaboration 사람·AI 협업

3 **계획(Plan)**
AI가 아키텍처 옵션과 각각의 장단점을 제시하고, 아키텍트와 개발자가 최적안을 고릅니다. 전략적 판단은 반드시 사람이 내립니다.
옵션 AI 선택 사람

4 **구현(Construct)**
AI가 코드와 테스트를 생성하고, 만들어지는 도중에 피드백이 즉시 반영됩니다. 리뷰가 작성 완료 후가 아니라 실시간으로 일어납니다.
Mob Construction

5 **검증(Verify)**
결과가 스펙과 일치하는지 대조합니다. 어긋나면 코드를 땜질하지 않고 스펙으로 돌아갑니다. 가장 자주 생략되면서 가장 중요한 단계입니다.
스펙 대조

개념

인간과 AI의 역할 분담

어디까지가 사람의 결정이고 어디부터가 AI의 실행인지가 이 방법론의 핵심입니다

단계	사람이 결정	AI가 실행
의도	무엇을 왜 만드는가(비즈니스 목표)	해당 없음 — 사람 고유 영역
스펙	가정·제약·성공 기준 승인	스펙 초안 작성, 빈틈 질문
계획	아키텍처·기술 스택 최종 선택	옵션 제시와 장단점 분석
구현	방향 조율, 실시간 피드백	코드·테스트 생성
검증	통과·반려 판정, 스펙 복귀 결정	스펙 대비 정합성 점검, 테스트 실행

> 한마디로 **판단·목표·승인은 사람, 생성·조사·반복은 AI**입니다.

AWS, 'AI-Driven Development Life Cycle' (AWS DevOps Blog); re:Invent 2025 DVT214

심화

멀티 에이전트 검증

AI는 제 코드에 관대하므로, 역할이 다른 에이전트로 견제합니다

단독 에이전트

자기 검증의 관대함

- ✕ 성공할 케이스만 골라 테스트에 넣습니다
- ✕ 자기 코드를 리뷰하면 문제를 잘 못 찾습니다
- ✕ 완료로 표시됐지만 실제 구현은 70%인 결과가 나옵니다

VS

멀티 에이전트

비판적 상호 검증

- ✓ 작성 에이전트와 검증 에이전트를 분리합니다
- ✓ 엣지·에러 케이스를 강제로 점검합니다
- ✓ 어긋남을 조기에 발견해 단독 실행보다 품질이 높습니다

원칙: **검증에 토큰을 아끼지 마라.** 잘못된 완료를 믿고 프로덕션에 올리는 비용이 훨씬 큽니다.

프렌티스 KOSTA AI Driven Development 강의 현장 관찰 (AI 자기 관대성·verify 미실행 시 70% 구현 사례); AI-DLC 멀티 에이전트 검증 원칙

예제

로그인 기능 적용 예시

3회 실패 잠금은 제약이 되고, 세션과 JWT는 사람이 선택합니다

단계	실제 벌어지는 일
의도	PM이 안전한 로그인 기능이 필요하다는 목표를 제시합니다
스펙	AI가 비밀번호를 틀리면 어떻게 하는지 질문하고, 팀이 3회 실패 시 계정 잠금이라 답합니다
계획	AI가 세션 방식과 JWT 방식을 제시하고, 아키텍트가 JWT를 선택합니다
구현	AI가 인증 코드와 테스트를 생성하고, 개발자가 잠금 해제 흐름 누락을 즉시 지적합니다
검증	다른 에이전트가 3회 실패 시나리오의 테스트 존재를 대조하고, 없으면 스펙으로 돌아갑니다

> 의도와 계획 선택은 **사람**이, 초안과 코드와 검증 실행은 **AI**가 맡는 경계가 한눈에 보입니다.

개념

개발자 역할의 이동

직접 짜는 시간은 줄고, 정의하고 조율하고 검증하는 시간이 늘어납니다

Orchestrator

Coder에서의 이동

코드를 치는 사람에서, 방향을 정의하고 여러 AI를 조율하며 결과를 판정하는 사람으로 무게 중심이 옮겨집니다.

1 **정의하는 시간이 늘어납니다**
무엇을 왜 만들지 스펙으로 명확히 정의하는 일이 개발자의 앞단 업무가 됩니다.

2 **조율하는 시간이 늘어납니다**
서로 다른 역할의 여러 에이전트를 붙였다 떼며 작업을 배분하고 견제시킵니다.

3 **검증하는 시간이 늘어납니다**
AI가 완료로 표시한 결과를 스펙과 대조해 통과와 반려를 판정합니다.

4 **소규모 교차기능팀이 유리합니다**
AI-DLC는 4~6명 Taxi Team을 권장합니다. 커뮤니케이션 복잡도는 인원의 제곱에 비례해, 10명이면 채널 45개, 5명이면 10개입니다.

LO-AI-DLC 생명주기와 인간·AI 역할 (Frentis 2026); AI-DLC Taxi Team(4~6명 교차기능팀), 커뮤니케이션 복잡도 n(n-1)/2

설명

직무별 역할 변화

PM은 퍼실리테이터로, 개발자는 검증자와 의사결정자로, QA는 커버리지 판정자로 무게가 옮겨집니다

역할	이전	AI-DLC에서
PM·기획자	요구사항 문서를 단독으로 작성한 뒤 전달	스펙 협업 세션의 퍼실리테이터, AI 질문에 비즈니스 관점으로 답변
개발자·아키텍트	코드 작성자	코드 검증자, 아키텍처 의사결정자, AI가 놓친 엣지 케이스 발굴
QA	테스트 코드 작성	테스트 전략 수립, AI 생성 테스트의 커버리지 판정

공통 방향은 **작성자에서 판정자로**의 이동입니다. 무엇을 만들지 정의하고, 결과가 스펙과 맞는지 판단하는 일이 중심이 됩니다.

심화

도입 전 주의점

갓 나온 방법론이라, 조직에 맞게 조정할 판단이 필요합니다

⚠ WARNING **실전 검증이 아직 부족합니다**

2025년 8월에 발표된 새 방법론이라 대규모·장기 운영에서의 축적된 검증이 부족합니다.

초기 도입은 작은 범위에서 시작해 효과를 확인한 뒤 확장하는 편이 안전합니다.

⚠ WARNING **실시간 협업을 전제합니다**

Mob Elaboration과 Mob Construction은 함께 모여 진행하므로, 시차 큰 분산 팀에는 적응 비용이 큽니다.

🚫 ERROR **규제 산업과 레거시에서는 과제가 남습니다**

금융·의료처럼 고도로 규제된 영역이나 복잡한 레거시 유지보수에서는 AI 생성 코드의 감사 가능성과 맥락 이해가 과제로 남습니다.

방법론을 이식하기보다 우리 조직 상황에 맞게 조정하는 판단이 필요하며, 그 판단 역시 사람의 몫입니다.

AI-DLC 비판적 분석, medium.com/data-science-collective (실전 검증 부족, 규제 산업 한계); AWS AI-DLC 발표 시점 기준

레퍼런스

AI-DLC 방법론의 1차 출처(AWS)와 핵심 개념, 강의 현장 관찰, 선행 LO의 정확한 출처입니다

방법론 · AWS

AI-DLC

AWS가 발표한 AI 중심 개발 방법론입니다. 핵심 원칙은 "AI proposes, human approves"이며, 3단계(Inception·Construction·Operation)와 Mob Elaboration·Mob Construction으로 진행합니다.

예시: aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/ · re:Invent 2025 DVT214

재정의 단위

Bolt · Units of Work

2~4주 Sprint를 시간·일 단위 Bolt로, 추상적 Epic을 AI가 한 번에 처리하는 Units of Work로 재정의합니다. AWS는 프로젝트 기간이 8~12주에서 2~3주로 줄었다고 밝혔습니다.

예시: AWS DevSphere 2025; Wipro CTO — 프로덕션 모듈 4개를 20시간에 구축

조직 구조

Taxi Team

4~6명 규모의 소규모 교차기능팀을 권장합니다. 커뮤니케이션 복잡도가 인원의 제곱에 비례하므로(n(n-1)/2) 실시간 협업에는 작은 팀이 유리합니다.

예시: 10명 = 채널 45개, 5명 = 채널 10개

Frentis

현장 관찰 · 선행 LO

AI 자기 관대성과 verify 미실행 시 70% 구현 사례는 프렌티스 강의 현장 관찰입니다. AI-DLC는 앞서 배운 개발 방법론 스펙트럼 위에 놓입니다.

예시: 프렌티스 KOSTA AI Driven Development 강의; LO-AI 시대 개발 방법론 스펙트럼 (Frentis 2026)

LO2 · AI-DLC 생명주기와 인간·AI 역할

LO2 체크포인트

- ✔ AI-DLC의 다섯 단계(의도·스펙·계획·구현·검증)를 순서대로 나열하고 각 단계의 핵심 활동을 말할 수 있습니다
- ✔ 각 단계에서 사람이 결정하는 것과 AI가 실행하는 것을 구분해 설명하고, Bolt와 Sprint, Units of Work와 Epic의 차이를 말할 수 있습니다
- ✔ AI의 자기 관대성 문제가 왜 멀티 에이전트 검증으로 해결되는지, 어긋남이 나오면 코드가 아니라 스펙으로 돌아가는 이유는 무엇인지 설명할 수 있습니다
- ✔ 자신의 직무(개발·PM·QA)가 AI-DLC에서 어떻게 바뀌는지 구체적으로 말할 수 있습니다

SECTION

03 SDD 원리 — 의도가 진실의 원천

- "의도가 진실의 원천"이 전통 개발의 무엇을 뒤집는지, 버그의 정의가 어떻게 달라지는지 설명합니다
- ACME(Assumptions·Constraints·Metrics·Examples)로 모호한 요구를 검증 가능한 스펙으로 다듬습니다
- 스펙을 사람과 AI 사이의 계약으로 보고, Given-When-Then으로 성공·실패 경로를 함께 못박습니다
- "AI는 자기 코드에 관대하다"는 편향을 이해하고, 스펙을 AI 바깥 기준으로 삼는 독립 검증을 논증합니다

50분 (블록 3)

개념

진실의 원천 전환

전통 개발은 코드가 진짜였지만, SDD는 스펙이 진짜이고 코드는 그 표현입니다

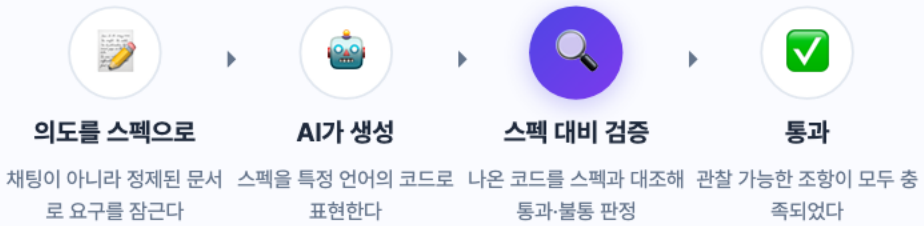
구분	코드 중심 (전통)	의도 중심 (SDD)
진실의 원천	코드가 진짜, 문서는 사후 설명	의도(스펙)가 진짜
문서 시점	코드를 짠 뒤에 문서를 씀	스펙이 코드를 생성한다
버그의 정의	코드가 잘못됨	스펙과 코드가 어긋남
AI의 역할	검색·자동완성 도구	스펙을 구현하는 협업 파트너

코드에 버그가 있다면 그것은 **스펙을 잘못 구현**한 것이지 스펙이 틀린 것이 아닙니다. 이 전환은 말장난이 아니라 작업 순서와 검증 기준을 함께 바꿉니다.

설명

SDD의 피드백 루프

검증에 실패하면 코드를 손대지 않고, 스펙으로 돌아가 다시 생성합니다



↩ 검증 실패 시: 코드를 땜질하지 않고 스펙으로 되돌아가 의도를 다듬고 다시 생성합니다 — 이 되돌림 고리가 SDD의 뼈대입니다.

vibe coding은 요구가 채팅 히스토리에 흩어지고 검증 기준이 없어 프로덕션에서 무너집니다. SDD는 요구를 **문서로 잠가(lock)** 이 흩어짐을 막고, 스펙 대비 검증으로 품질 게이트를 세웁니다.

예제

의도가 빠진 코드

validateProductInput 코드는 무엇을 하는지 보여줄 뿐, 왜 그렇게 정했는지는 남기지 않습니다

products.ts typescript

```typescript
if (!partial || provided('price')) {
  if (!Number.isInteger(input.price) || input.price < 0) {
    return 'price must be a non-negative integer (cents)'
  }
}
if (!partial || provided('stock')) {
  if (!Number.isInteger(input.stock) || input.stock < 0) {
    return 'stock must be a non-negative integer'
  }
}
```

src/products.ts · validateProductInput

코드가 답하지 못하는 질문

- price는 왜 정수여야 하는가 — 단위가 원인가 센트인가?
- 왜 음수를 400으로 막는가 — 재고 0은 허용인가?
- 이미 삭제된 상품을 다시 수정하면 어떻게 되어야 하는가?
- 무효 수정이 들어오면 저장된 상품은 그대로여야 하는가?

답은 **스펙**에만 있습니다. 스펙이 없으면 이 결정들은 작성자의 머릿속에 갇히고, AI는 빈칸을 임의 가정으로 메웁니다.

실측: openspec-ecommerce-api/src/products.ts validateProductInput

설명

ACME — 의도 검증의 네 기둥

"정확해야 한다"는 스펙이 아니라 소원입니다. Assumptions·Constraints·Metrics·Examples는 모호한 요구를 검증 가능한 스펙으로 다듬는 네 점검 축입니다

요소	의미	핵심 질문	빠지면
A · Assumptions	가정	당연하게 깔고 적지 않은 전제는 무엇인가	AI가 임의 가정을 만든다
C · Constraints	제약	지켜야 할 한계와 경계는 무엇인가	자유도가 안전하게 줄지 않는다
M · Metrics	지표	성공을 어떻게 객관적으로 측정하는가	주관적 판단만 남는다
E · Examples	예시	구체적 입력과 출력은 어떤 모습인가	"보여주기"가 없어 오해가 는다

ACME는 작성 프레임이자 검토 체크리스트입니다. 스펙을 쓴 뒤 **A·C·M·E 네 축이 다 채워졌는지** 되짚으면 빈 구석이 드러납니다. 특히 Examples는 가장 강력하지만 자주 빠지는 요소입니다.

예제

ACME 적용 전후

'삭제가 잘 되어야 한다'는 검증할 수 없지만, 네 축으로 다듬으면 관찰 가능한 조항이 됩니다

적용 전 — 소원(wish)

spec (before) markdown

```
# Feature: Todo 삭제 API
- 사용자가 할 일을 삭제할 수 있어야 한다
- 삭제가 잘 되어야 한다
- 빠르게 동작해야 한다
```

"잘", "빠르게"는 판정 기준이 없어 AI가 빈칸을 스스로 채웁니다.

적용 후 — 검증 가능한 계약

spec (after · ACME) markdown

```
## Assumptions
- 사용자는 인증된 상태(JWT 필수), Todo ID는 UUID v4
## Constraints
- 응답 p95 200ms 이하, 자신의 Todo만 삭제 가능
- 이미 삭제된 Todo 재삭제 시 404
## Metrics
- 권한 없는 요청 차단률 100%, 정상 삭제 성공률 99.9% 이상
## Examples
- 정상 삭제 → 200, 권한 없음 → 403, 미존재 → 404
```

모든 조항이 상태 코드·수치로 관찰 가능해, 그대로 테스트가 됩니다.

출처: LO-Spec-Driven Development ACME 절 (CU-AI Driven Development-KOSTA)

개념

계약으로서의 스펙

스펙은 AI에게 지킬 조건을, 사람에게 판정할 기준을 정해 주는 계약입니다

규율 ① — What/Why만, How는 위임

무엇(What)과 왜(Why)만 쓰고 어떻게(How)는 구현자에게 넘깁니다. 구현 방법까지 못박으면 AI의 강점을 죽이고, 스펙이 코드와 중복되어 유지비만 늘어납니다.

규율 ② — 모든 조항은 관찰 가능

겉으로 확인할 수 없는 조항은 검증할 수 없으니 계약 효력이 없습니다. "가격을 음수로 수정하면 거부(400)하고 상품은 변경되지 않는다"는 관찰 가능하지만, "적절히 처리한다"는 그렇지 않습니다.

계약이 명확하면 버그 논쟁이 줄고, 다툼이 생기면 **계약서(스펙)**로 되돌아가 시비를 가립니다.

ℹ️ INFO **SDD 성숙도의 방향**

조직은 필요할 때만 스펙을 먼저 쓰는 단계에서 시작해, 스펙이 기능 생애 전체에 걸쳐 유지되는 살아있는 계약 단계로 발전합니다.

이상은 스펙만 사람이 편집하고 코드는 그 컴파일 결과인 단계지만, 현실적 목표는 "스펙이 살아있는 계약으로 유지되는" 단계입니다.

💡 TIP **계약이 규율을 만든다**

스펙을 "잘 쓴 문서"가 아니라 계약으로 보면, 무엇을 적고 무엇을 빼야 하는지가 저절로 분명해집니다.

구현 세부는 계약에 넣지 않습니다. 그것은 이행 방법이지 합의 조건이 아니기 때문입니다.

설명

Given-When-Then 구조

상태·행위·결과를 한 문장으로 못박고, 실패 경로도 성공과 같은 엄밀함으로 적습니다

1 **관찰 가능한 결과**

Then은 상태 코드·응답 본문·저장 상태처럼 밖에서 확인 가능한 사실이어야 합니다. 인상이 아니라 사실로 판정합니다.

2 **실패 경로의 명시**

정상 케이스만 적으면 AI는 에러 처리를 임의로 만듭니다. 무효 입력(400), 미존재(404), 충돌(409)을 성공과 같은 비중으로 적습니다.

3 **부작용의 못박음**

"거부한다"에 더해 "그리고 상품은 변경되지 않는다"처럼 실패가 저장 상태를 오염시키지 않음을 조항으로 남깁니다.

이 규율이 지켜지면 **시나리오 하나가 곧 테스트 하나**로 대응되어, 스펙과 테스트가 자연스럽게 추적됩니다.

예제

실제 spec — Update Product

하나의 SHALL 아래 정상·미존재·무효 시나리오와 부작용까지 못박은 실제 spec입니다

spec.md — markdown

```
### Requirement: Update Product
The system SHALL apply a partial update to an existing
product's `name`, `price`, or `stock`, enforcing the same
validation as creation, or `404` when it does not exist.

#### Scenario: Update existing product
- **WHEN** a client sends `PATCH /products/{id}` with { "price": 3900 }
- **THEN** the system responds `200`
- **AND** the returned product reflects the new price

#### Scenario: Update missing product
- **WHEN** a client sends `PATCH /products/{id}` for a missing id
- **THEN** the system responds `404`

#### Scenario: Reject invalid update
- **WHEN** a client sends `PATCH /products/{id}` with price < 0
- **THEN** the system responds `400`
- **AND** the product is left unchanged
```

specs/product-catalog/spec.md

SHALL

requirement 본문은 SHALL로 의무를 진술합니다. "할 수도 있다"가 아니라 "해야 한다"입니다.

실패도 같은 비중으로

정상 수정과 함께 Update missing(404)·Reject invalid(400)가 나란히 붙습니다. 실패 경로를 못박는 것이 계약의 핵심입니다.

left unchanged

무효 수정이 저장 상태를 오염시키지 않음을 조항으로 남깁니다. 부작용까지 관찰 가능하게 적었습니다.

Scenario

hashtag는 정확히 4개입니다. 이 형식만 delta 파서가 시나리오로 인식합니다 — 형식이 곧 계약의 유효 요건입니다.

실측: openspec-ecommerce-api/openspec/specs/product-catalog/spec.md

심화

AI 자기검증의 한계

AI에게 제 코드를 검토시키면 제 출력을 정답으로 여깁니다. 판정 기준은 AI 바깥, 곧 스펙에 둡니다

⚠️ WARNING **검증자의 편향**

AI는 스스로 세운 가정을 스스로 심사하니 통과가 잘 됩니다. 게다가 에러 없이 실행되면 품질이 괜찮다고 넘기는 태도가 흔합니다.

이 방식으로 논리 버그와 보안 결함이 잠복하는 사례가 반복됐습니다.

💡 TIP **판정 기준을 AI 바깥에**

검증은 "AI야, 잘했니"가 아니라 "이 코드가 스펙의 모든 시나리오를 통과하는가"를 묻습니다.

시나리오가 관찰 가능하게 적혀 있으면, 판정은 사람의 인상이 아니라 실행 결과가 내립니다.

바깥 기준의 실물 — Success Criteria

spec.md · Spec Kit — markdown

```
## Success Criteria
- SC-002: 모든 무효 입력 경로는 400, 모든 미존재 경로는
          404를 반환하고, 정상 조건에서 500을 반환하지 않는다.
- SC-004: 무효 수정은 저장된 상품을 절대 변경하지 않는다.
```

specs/001-product-catalog/spec.md

SC-002·SC-004는 실제로 실행해 통과를 판정합니다. 무효 입력이 400을 내는가, 무효 수정이 저장 상태를 바꾸지 않는가를 **돌려봅니다** — AI에게 물어보지 않습니다.

개념

SDD 방어 메커니즘

계약·ACME·독립 검증 세 겹이 AI의 속도와 품질을 함께 지킵니다

01 계약

의도를 스펙으로 잠근다

What/Why만 쓰고 모든 조항을 관찰 가능하게. 요구가 채팅에 흩어지지 않고 문서 한 곳에 고정됩니다.

02 ACME

빈칸을 없앤다

Assumptions·Constraints·Metrics·Examples로 AI가 임의 가정으로 메울 여지를 지웁니다.

03 독립 검증

AI 바깥 기준으로 판정한다

Given-When-Then 시나리오를 실제로 실행해 통과·불통을 가립니다. 자기관대 편향을 스펙이 차단합니다.

레퍼런스

SDD 원리의 개념적 뿌리와, 원리가 실제로 구현된 두 산출물(OpenSpec·Spec Kit)의 정확한 출처입니다

원리 · 선행 개념

SDD

Spec-Driven Development. 의도(스펙)를 진실의 원천으로 삼아 코드를 그 결과물로 취급하는 방법론입니다. 검증 실패는 스펙으로 회귀합니다.

예시: LO-SDD 원리·ACME·Todo API 예제 (CU-AI Driven Development-KOSTA)

구현체 · Fission-AI

OpenSpec

SDD를 파일과 CLI로 실현한 도구입니다. Requirement의 SHALL 진술과 #### Scenario로 계약을 기술합니다.

예시: github.com/Fission-AI/OpenSpec · specs/product-catalog/spec.md

구현체 · GitHub

Spec Kit

Constitution→Specify→Plan→Tasks→Implement 워크플로우로 각 단계를 Markdown 산출물로 남깁니다. Success Criteria가 AI 바깥 판정 기준입니다.

예시: github.com/github/spec-kit · specs/001-product-catalog/spec.md (SC-002·SC-004)

SDD 맥락

선행·후속 LO

AI-DLC 생명주기가 의도 우선 위임을 규정하면, SDD가 그 의도를 어떻게 기록할지 다룹니다. 산출물 지도로 이어집니다.

예시: 선행: LO-AI-DLC 생명주기와 인간·AI 역할 / 후속: LO-SDD 산출물 지도

체크포인트

스스로 답할 수 있으면 LO3를 통과한 것입니다. 진실의 역전부터 독립 검증까지 다섯 가지를 점검합니다

01 진실의 원천 역전

SDD는 코드가 아니라 스펙을 정보원으로 삼습니다. 버그의 정의가 "코드가 잘못됨"에서 "스펙과 코드가 어긋남"으로 바뀝니다.

02 ACME로 다듬기

모호한 요구 한 줄을 Assumptions·Constraints·Metrics·Examples 네 축으로 다듬어 검증 가능한 스펙으로 바꿉니다.

03 스펙 = 계약

What/Why만 쓰고 How는 위임, 모든 조항은 관찰 가능. 이 두 규율이 계약으로서의 스펙을 지탱합니다.

04 실패 경로와 부작용

Given-When-Then에서 실패 경로(400·404·409)와 부작용("변경되지 않는다")을 성공과 같은 비중으로 적습니다.

05 독립 검증

"AI는 자기 코드에 관대하다"는 편향 때문에, 판정 기준을 AI 바깥의 스펙에 두고 시나리오를 실행해 판정합니다.

🎯 코드가 아니라 스펙이 정보원입니다. 검증에 실패하면 코드를 땜질하지 않고 스펙으로 되돌아갑니다.

SECTION

04 SDD 산출물 지도 — proposal · spec · design · tasks

- SDD 산출물 4종이 전통 SE 산출물(PRD·SRS·ADR·WBS)의 어디에 대응하는지 지도로 설명합니다
- 각 산출물의 역할·주 독자·검증 기준을 구분하고, SDD가 왜 이 DAG 순서를 강제하는지 근거를 듭니다
- 추적성(spec ↔ code ↔ test)이 어떤 링크로 성립하고 어디서 끊기는지 짚습니다
- OpenSpec과 Spec Kit이 같은 개념을 서로 다른 파일로 실현하는 대응표를 읽습니다

50분 (블록 4)

개념

산출물 지도의 필요성

두 도구는 이름이 달라도, 전통 소프트웨어 공학이 다룬 같은 네 질문에 답합니다

왜 만드나	정확히 무엇을	어떻게 만드나	어떤 순서로
제안 (proposal)	요구 (spec)	설계 (design/plan)	작업 (tasks)

01
처음 보는 도구를 빠르게 해석
"이건 PRD구나, 이건 ADR이구나"로 대응시키면 학습 비용이 줄어듭니다.

02
산출물의 역할 혼동 차단
설계 결정을 요구사항에 적거나, 작업 목록에 요구사항을 뒤섞는 실수를 지도가 걸러냅니다.

03
검증 기준을 산출물별로 분리
"이 파일은 무엇으로 통과를 판정하는가"를 산출물마다 또렷하게 정합니다.

설명

산출물 지도

SDD 산출물은 전통 소프트웨어 공학의 산출물을 AI 협업에 맞게 다시 짠 것입니다

SDD 산출물	전통 SE 대응	답하는 질문	주 독자	검증 기준
proposal	PRD (제품 요구 정의)	왜·무엇을 바꾸나	사람 (리뷰어·PO)	범위·비목표가 명확한가, 영향 파일이 식별됐나
spec	상세 요구사항 (SRS)	정확히 무엇을 만드나	사람 + AI	시나리오가 관찰 가능·실패 경로 포함인가
design / plan	설계 문서 + ADR	어떻게 만드나	AI 구현자 + 사람	결정·기각 대안·기술 제약이 근거와 함께 있나
tasks	WBS (작업 분해 구조)	어떤 순서로 쪼개나	AI 실행자 + 사람	각 작업이 검증 가능한 크기·의존 순서가 맞나

예제

proposal ↔ PRD, spec ↔ SRS

proposal은 범위와 비목표로 경계를 긋고, spec은 그 안에서 요구와 검증 기준을 정합니다

proposal ↔ PRD — 범위와 비목표

proposal.md (OpenSpec, 실물) — markdown

```
## What Changes
- 상품 자원 REST API 신설
  (POST / GET / PATCH / DELETE)
- 입력 검증 위반 시 400, 미존재 404

## Non-goals
- 주문 생성·재고 차감 → 별도 change
- 인증·인가, 영속 DB → 1막은 인메모리
- 페이지네이션·검색·정렬 필터
```

proposal-excerpt

범위와 비목표가 나란히 있어 이번 단위의 경계가 확정됩니다. 서명하는 주 독자는 리뷰어입니다.

spec ↔ SRS — FR와 검증 기준

spec.md (Spec Kit, 실물) — markdown

```
## Functional Requirements
- FR-005: 상품의 name·price·stock을
  생성과 같은 검증으로 부분 수정(200),
  미존재 404, 무효 수정(400)은 상품을
  변경하지 않는다.

## Success Criteria
- SC-004: 무효 수정은 저장된 상품을
  절대 변경하지 않는다.
```

spec-excerpt

FR이 '무엇을'을 못박고 Success Criteria가 검증 기준을 세웁니다. spec은 구현 방법을 적지 않습니다.

실물 발췌: openspec-ecommerce-api proposal.md · speckit-ecommerce-api specs/001-product-catalog/spec.md

예제

design ↔ ADR, tasks ↔ WBS

design은 기각한 대안까지 근거로 남기고, tasks는 커밋 하나 크기로 테스트부터 배치합니다

design ↔ ADR — 결정과 기각

design.md (OpenSpec, 실물) markdown

```markdown
## Decisions
### 에러 모델
- 검증 실패 400, 미존재 404.
  본문 { "error": string }.

### Rejected Alternatives
- zod 검증: 1막 스코프에 과하고
  의존성 증가 → 수동 검증으로 시작
- 클래스 기반 도메인 모델: 순수 데이터
  + 함수형 store로 충분 → 과설계 회피
```

design-excerpt

채택 결정만이 아니라 기각한 대안과 그 이유를 남기는 것이 ADR의 핵심입니다.

tasks ↔ WBS — 검증 가능·의존 순서

tasks.md (Spec Kit, 실물) markdown

```markdown
## Phase 3: User Story 1 (P1)
- [x] T005 [P] [US1] POST /products
  요청 테스트 작성 (201, 400×3)
- [x] T006 [US1] POST /products 구현
- Checkpoint: US1 독립 테스트 가능

## Dependencies
- [P] = 다른 파일이라 병렬(주로 테스트)
  같은 파일 구현은 순차
```

tasks-excerpt

각 작업이 커밋 하나 크기이고, [US1] 태그가 spec의 User Story 1로 역추적되며, 테스트가 구현 앞에 옵니다.

실물 발췌: openspec-ecommerce-api design.md · speckit-ecommerce-api specs/001-product-catalog/tasks.md

설명

산출물 의존 순서

산출물은 임의 순서가 아니라 의존 그래프로 묶여, 부모 없이 자식을 만들 수 없습니다



OpenSpec

proposal → (spec, design) → tasks. applyRequires: [tasks] — 구현하려면 tasks까지 done이어야 합니다.

Spec Kit

constitution → spec → plan → tasks → implement의 선형 게이트. 각 단계가 다음 단계의 입력이 됩니다.

공통 원리

각 단계가 앞 단계를 근거로 삼도록 묶어, 아래 단계의 결정이 위 단계로 소급 검증되게 만듭니다.

실측: openspec 1.5.0, openspec status --change product-catalog (작성 전 시점, 0/4)

심화

의존 순서의 강제

단계를 건너뛰면 경계·근거·구조가 무너집니다. tasks 완료가 사람의 서명 지점입니다

⚠️ WARNING **제안 없이 요구를 쓰면**

경계 없는 요구가 쏟아집니다. proposal의 범위·비목표가 없으니 '무엇을 안 할지'가 흐려집니다.

⚠️ WARNING **요구 없이 설계하면**

'무엇을 위한 설계인지' 근거를 잃습니다. spec이라는 계약 본문 없이 결정이 공중에 뜹니다.

ℹ️ INFO **tasks 완료 = 사람의 서명 지점**

tasks가 다 끝난 시점이 propose와 apply 사이의 사람 리뷰 지점입니다.

"계약서에 서명하기 전에 읽어라" — 여기서 사람이 의도서 전체를 검토하고 승인합니다.

설명

추적성

어느 코드가 어느 요구를 구현하고 어느 테스트가 지키는지, 세 링크로 잇습니다

링크 ①

spec → test

각 시나리오(Given-When-Then)가 테스트 하나로 대응됩니다. 시나리오가 관찰 가능하게 적혀 있으므로 기계적으로 테스트로 옮겨집니다.

> **예시:** "무효 수정은 상품 불변" 시나리오 → SC-004 → 400 leaves unchanged 테스트

링크 ②

spec → code

각 요구(Requirement/FR)가 구현 단위(라우트·함수)로 대응됩니다. tasks가 이 대응을 작업 항목으로 명시합니다.

> **예시:** FR-005(상품 부분 수정) → T010(테스트)·T011(구현)

링크 ③

test → code

테스트가 코드의 동작을 고정합니다. 테스트를 먼저 쓰는 순서(tasks의 배치)가 이 링크를 앞세웁니다.

> **예시:** T005(테스트) → T006(구현) 순서로 [P] 태그가 병렬 테스트를 표시

예제

요구사항 추적 흐름

요구 하나가 proposal에서 코드까지 관통하고, 사슬이 닫히면 추적성이 성립합니다

링크	이 예제에서
proposal → spec	"상품 수정" 범위 → FR-005 (생성과 같은 검증으로 부분 수정)
spec → task	FR-005 → T010(테스트) · T011(구현)
spec → test	"무효 수정은 상품 불변" 시나리오 → SC-004 → 400 leaves unchanged 테스트
design → code	"수동 검증 순수 함수" 결정 → validateProductInput(input, { partial })

한 요구가 모든 산출물을 관통합니다. 이 사슬이 닫혀 있으면 추적성이 성립하고, **끊긴 고리가 verify의 점검 대상**입니다.

실물: speckit-ecommerce-api specs/001-product-catalog (FR-005 · SC-004 · T010/T011 · validateProductInput)

심화

추적 단절과 검증

끊긴 링크는 대개 스펙 없는 코드나 테스트 없는 시나리오이며, 3C로 찾습니다

🚫 ERROR **끊기는 곳 — spec 없는 코드**

요구 없이 생긴 기능입니다. 코드에서 거슬러 올라갈 시나리오가 없어 사슬이 끊깁니다.

🚫 ERROR **끊기는 곳 — 테스트 없는 시나리오**

관찰 불가능하게 적힌 조항입니다. 시나리오가 테스트로 옮겨지지 못해 사슬이 끊깁니다.

💡 TIP **verify의 3C 점검**

OpenSpec은 세 축으로 끊긴 링크를 점검합니다.

완전성: 모든 작업이 완료됐나 · 정확성: 각 시나리오가 테스트로 존재하나 · 일관성: 설계 결정이 코드에 반영됐나

설명

두 구현의 산출물 대응

두 도구는 같은 개념 지도를 서로 다른 파일 구성으로 채웁니다

SDD 개념	OpenSpec	Spec Kit
왜·무엇을 바꾸나	proposal.md (Why/What/Impact/Non-goals)	spec.md의 User Scenarios
상세 요구사항	specs/ delta (Requirement + Scenario)	spec.md의 FR + Acceptance Scenario
설계	design.md 하나	plan.md + data-model.md + contracts/ + research.md
작업	tasks.md (무브 단위)	tasks.md (Phase · [P] · [US])
검증	verify 3C / validate --strict	quickstart E2E + checklist / analyze
거버넌스	config.yaml의 context·rules	constitution.md (단계 게이트)

실측: openspec-ecommerce-api/docs/steps/01, speckit-ecommerce-api/docs/openspec-vs-speckit.md · OpenSpec 1.5.0, Spec Kit(specify) 0.12.8

심화

두 구현의 핵심 차이

실질적 차이는 두 곳입니다 — 요구를 담는 단위와 설계 파일 수입니다

01 요구를 담는 단위

OpenSpec은 변경분만 적는 delta(ADDED/MODIFIED/REMOVED)로 기존 스펙에 병합하는 브라운필드 지향입니다. Spec Kit은 번호 붙은 feature 폴더에 전체 명세를 두는 그린필드 지향입니다.

02 설계를 담는 파일 수

OpenSpec은 design.md 하나에 결정을 모읍니다. Spec Kit은 설계를 data-model·contracts·research로 쪼갭니다. 지도에서는 둘 다 같은 '설계 산출물'이며 나뉜 파일 수만 다릅니다.

03 OpenSpec = 연속성 레이어

하나의 living spec을 델타로 진화시킵니다. 브라운필드·증분 변경에 강합니다. 변경은 새 feature가 아니라 기존 requirement의 MODIFIED로 다룹니다.

04 Spec Kit = 거버넌스 레이어

기능별 upfront 종합 명세(spec/plan/tasks)와 clarify·analyze 품질 게이트를 둡니다. 그린필드·구조적 진행에 강합니다.

🗺️ 이름·파일 수가 달라도 지도의 칸은 같습니다. 처음 보는 SDD 도구도 '이 파일은 전통의 무엇에 해당하고 누가 읽고 무엇으로 통과를 판정하는가'로 해석하면 됩니다.

실측: speckit-ecommerce-api/docs/openspec-vs-speckit.md (연속성 레이어 vs 거버넌스 레이어)

LO4 · SDD 산출물 지도

레퍼런스

산출물 지도가 대응시키는 두 구현체(OpenSpec·Spec Kit)의 1차 출처와, 관통 예제로 쓴 두 repo, 선행 LO의 정확한 출처입니다

구현체 · Fission-AI

OpenSpec

SDD 산출물 지도를 delta 기반 파일과 CLI로 실현한 도구입니다. 핵심 개념은 spec delta로, change마다 requirement 변화를 캡처합니다.

예시: github.com/Fission-AI/OpenSpec · openspec.dev · npm @fission-ai/openspec v1.5.0 (Node 20.19+)

구현체 · GitHub

Spec Kit

Constitution → Specify → Plan → Tasks → Implement 워크플로우로 각 단계가 Markdown artifact를 산출하는 도구입니다. specify CLI가 스캐폴드합니다.

예시: github.com/github/spec-kit · github.github.com/spec-kit · specify(0.12.8)

살아있는 교재

관통 예제 repo

같은 상품 CRUD 기능을 두 도구로 각각 개발한 실제 repo입니다. 산출물 발췌·대응표·커밋이 모두 이 두 repo의 실측입니다.

예시: openspec-ecommerce-api · speckit-ecommerce-api (docs/openspec-vs-speckit.md)

SDD 원리

선행 LO

산출물 지도가 놓이는 토대입니다. "의도가 진실의 원천"이라는 원리 위에 proposal·spec·design·tasks의 배치가 성립합니다.

예시: LO-SDD 원리 의도가 진실의 원천 (Frentis 2026)

LO4 · SDD 산출물 지도

체크포인트 — 스스로 점검

- ✔ proposal·spec·design/plan·tasks가 각각 전통 SE의 무엇에 대응하는지, 주 독자와 검증 기준을 구분해 말할 수 있다
- ✔ SDD가 산출물을 DAG로 강제하는 이유와, tasks 완료 시점이 왜 사람의 서명 지점인지 설명할 수 있다
- ✔ spec ↔ code ↔ test 추적성이 어떤 링크로 성립하고, 어디서(스펙 없는 코드·테스트 없는 시나리오) 끊기는지 짚을 수 있다
- ✔ OpenSpec의 design.md 하나와 Spec Kit의 data-model·contracts·research가 지도에서 같은 칸임을 설명할 수 있다
- ✔ 실제 산출물 발췌에서 각 파일이 어느 개념에 해당하는지 즉시 분류할 수 있다

SECTION

05 OpenSpec 기초 — artifact-guided 사이클

- OpenSpec 1.5.0을 설치하고 openspec init --tools claude로 프로젝트를 비대화형 초기화합니다
- status → instructions → 작성의 artifact-guided 워크플로우로 순서를 CLI가 알려주게 합니다
- 한 change의 전체 사이클(propose → apply → archive)을 상품 CRUD API로 직접 밟습니다
- context·rules를 산출물 파일에 넣지 않는 원칙을 지키고, git 태그로 임의 단계를 재현합니다

50분 (블록 5)

개념

OpenSpec의 위치

방법론·원리·구현체 세 층위를 구분하면, 도구가 바뀌어도 원리는 남습니다



AI-DLC (방법론)

AI가 실행하고 팀이 감독하는 상위 생명주기입니다. 슬로건은 AI proposes, human approves.

SDD (원리)

의도가 진실의 원천이라는 명세 원칙입니다. 결과물을 스펙과 대조해 검증합니다.

OpenSpec (구현체)

그 의도를 proposal · specs · design · tasks 네 산출물로 쪼개 파일과 CLI로 실현한 도구입니다.

🔑 OpenSpec은 AI-DLC의 구현체입니다. 도구가 바뀌어도 SDD 원리와 AI-DLC 방법론은 그대로 남습니다.

AI-DLC: AWS DevOps Blog, AI-Driven Development Life Cycle (2025) · SDD: 선행 LO(의도가 진실의 원천)

설명

change와 네 산출물

작업 단위는 change이고, 그 안에 proposal·specs·design·tasks를 만듭니다

artifact	답하는 질문	실체
proposal	왜·무엇을 바꾸는가	Why / What Changes / Capabilities / Impact / Non-goals
specs	관찰 가능한 계약은 무엇인가	delta 형식 requirement와 scenario
design	어떻게 만들고 무엇을 기각했는가	데이터 모델 · 라우트 표 · 에러 모델 · 기각 대안
tasks	어떤 순서로 구현하는가	TDD 순서의 체크박스 목록

설명

artifact-guided의 세 축

status가 순서를, instructions가 템플릿과 제약을 알려주고, 사람은 내용만 채웁니다

1 **openspec status — 순서와 완료 판정을 준다**

산출물 의존 그래프(DAG)를 보여줍니다. proposal 다음에 specs와 design, 그다음 tasks. 부모 없이 자식을 만들 수 없습니다. applyRequires: [tasks]는 구현 진입 게이트입니다.

`--change <n>` `--json`

2 **openspec instructions — 템플릿과 제약을 준다**

그 산출물의 template(빈 골격) · instruction(작성 지침) · context · rules · resolvedOutputPath(써야 할 위치)를 돌려줍니다.

`<artifact>` `--json`

3 **작성 — 받은 template에 내용만 채운다**

받은 골격을 내용으로 채웁니다. context와 rules는 채우는 사람과 AI의 제약으로만 쓰고, 산출물 파일에는 넣지 않습니다.

`template` `채우기`

예제

산출물 의존 그래프

openspec status가 의존 그래프를 보여줍니다. 처음엔 proposal만 열리고, 나머지는 부모가 없어 막혀 있습니다

```
openspec status --change product-catalog

- Loading change status...
Change: product-catalog
Schema: spec-driven
Change root: openspec/changes/product-catalog
Progress: 0/4 artifacts complete

[ ] proposal
[-] design (blocked by: proposal)
[-] specs (blocked by: proposal)
[-] tasks (blocked by: design, specs)
```

wf-03-status.txt · 0/4 complete

[ ] 는 지금 열린 산출물

proposal만 [ ]입니다. 부모가 없으니 바로 작성할 수 있는 유일한 산출물입니다.

[-] 는 blocked

design·specs는 proposal이, tasks는 design·specs가 끝나야 열립니다. 부모 없이 자식을 만들 수 없습니다.

DAG 순서

proposal → (specs, design) → tasks. 이 순서가 곧 작성 순서입니다.

Progress 0/4

완료 판정을 도구가 셉니다. 사람은 다음에 뭘 쓸지 고민할 필요가 없습니다.

실측: openspec 1.5.0, openspec status --change product-catalog (step-00-init 시점)

심화

구현 진입 게이트

status --json의 applyRequires는, 구현에 들어가려면 tasks까지 끝내라는 게이트입니다

openspec status --change product-catalog --json


```
"changeName": "product-catalog",
"isComplete": false,
"applyRequires": ["tasks"],
"nextSteps": [
  "Run openspec instructions proposal
   --change \"product-catalog\" --json"
],
"artifacts": [
  { "id": "proposal", "status": "ready" },
  { "id": "design",   "status": "blocked",
    "missingDeps": ["proposal"] },
  { "id": "specs",    "status": "blocked",
    "missingDeps": ["proposal"] },
  { "id": "tasks",    "status": "blocked",
    "missingDeps": ["design", "specs"] }
]
```


wf-04-status-json.txt (발췌)

applyRequires: [tasks]
apply(구현)에 들어가는 유일한 조건은 tasks 완료입니다. tasks가 끝나기 전엔 구현 단계로 넘어갈 수 없습니다.

nextSteps
도구가 지금 밟을 명령을 그대로 문장으로 줍니다. 여기서는 openspec instructions proposal을 먼저 실행하라고 안내합니다.

missingDeps
각 blocked 산출물이 무엇을 기다리는지 정확히 알려줍니다. tasks는 design과 specs 둘 다를 기다립니다.

isComplete: false
텍스트 status와 같은 사실을 기계가 읽는 형태로 노출합니다. 자동화·재현이 이 JSON을 소비합니다.

실측: openspec status --change product-catalog --json (wf-04-status-json.txt, step-00-init 시점)

예제

산출물 작성 지시서

instructions는 목적·지침·빈 골격을 태그로 돌려줍니다. JSON이 아니라 XML 형식입니다

openspec instructions proposal

```
<artifact id="proposal" change="product-catalog"
          schema="spec-driven">
<task>
  Create the proposal artifact for change
  "product-catalog".
</task>
<output>
  Write to: openspec/changes/product-catalog/proposal.md
</output>
<instruction>
  Create the proposal document that establishes
  WHY this change is needed.
  Sections: Why / What Changes / Capabilities / Impact
</instruction>
<template>
  ## Why ... ## What Changes ... ## Capabilities
</template>
<unlocks> Completing this enables: design, specs </unlocks>
</artifact>
```

wf-05-instr-proposal · openspec 1.5.0

<task> · <instruction>

이 산출물이 무엇을 위한 것인지, 어떤 절을 어떤 관점으로 채워야 하는지 자연어로 지시합니다. proposal은 how가 아니라 why에 집중하라고 못박습니다.

<template>

채워야 할 빈 골격입니다. Why / What Changes / Capabilities / Impact 헤더가 주석과 함께 들어 있고, 사람과 AI는 이 구조에 내용만 얹습니다.

<unlocks>

이 산출물을 완료하면 무엇이 열리는지 알려줍니다. proposal을 끝내야 design·specs로 진입할 수 있습니다 — status의 DAG와 짝을 이룹니다.

핵심: 이 출력은 **JSON이 아니라 XML 태그 형식**입니다. 자동화·파싱이 필요하면 `--json`을 붙여 구조화된 형태로 받습니다.

실측: openspec instructions proposal --change product-catalog (openspec 1.5.0)

심화

프로젝트 제약의 위치

제약은 config.yaml의 context·rules에 있고, 지시에 실려 오지만 산출물 파일엔 넣지 않습니다

context — 프로젝트 제약

기술 스택(TypeScript · Hono · pnpm · vitest), 저장 방식(1막은 인메모리, 인터페이스 뒤), 컨벤션(Conventional Commits · 파일당 단일 책임 · 명시적 HTTP 상태코드), 도메인 규칙을 담습니다.

rules — 산출물별 규칙

예: specs 규칙은 "모든 Requirement는 WHEN/THEN Scenario를 최소 하나 갖는다", "실패 시나리오(잘못된 입력·미존재·충돌)를 포함한다", "구현 세부가 아니라 관찰 가능한 API 동작을 기술한다".

instructions는 config.yaml의 이 둘을 읽어 지시에 실어 보냅니다. 그래서 산출물 지시에 섞여 오지만, 그것은 **채우는 사람(또는 AI)에게 제약을 주입**하기 위한 것입니다.

⚠ WARNING **함정 ① — 파일에 복붙 금지**

context/rules는 두뇌를 위한 제약이지 산출물이 아닙니다. proposal·spec 파일에 그대로 복사하면 스펙이 프로젝트 설정을 중복해 안고 커집니다.

규칙은 config.yaml 한 곳에만 둡니다. 산출물 파일은 그 규칙을 지킨 결과만 담습니다.

ℹ INFO **출처는 config.yaml 하나**

구버전의 project.md가 아니라 openspec/config.yaml의 context/rules로 통합됐습니다. 제약을 바꾸려면 여기를 고치고, 산출물은 손대지 않습니다.

커밋 2d186c6 chore(openspec): define project context and artifact rules 에서 최초 정의됩니다.

실습

설치와 초기화

openspec --version으로 1.5.0을 확인하고, npm i -g로 전역 설치한 뒤 openspec init --tools claude --force로 프로젝트를 비대화형 초기화합니다. Claude Code 통합 파일과 /opsx:* 슬래시 커맨드가 함께 깔립니다

bash

```
# 버전 확인 (2026-07-08 기준)
$ openspec --version
1.5.0
# 전역 설치 (또는 프로젝트에 --save-dev)
$ npm i -g @fission-ai/openspec@latest
# 비대화형 초기화 (헤드리스 안전)
$ openspec init --tools claude --force
✔ OpenSpec initialized (tools: claude)
```

version.txt · commit e94e129 (23 files, +4,109)

--tools claude
Claude Code용 통합 파일과 /opsx:* 슬래시 커맨드를 비대화형으로 설치

--force
legacy 파일을 프롬프트 없이 정리 (재현·재실행 안전)

초기화가 만드는 것

openspec/config.yaml
프로젝트 context / rules — 이후 모든 instructions가 여기서 제약을 읽어 옵니다.

.claude/commands/opsx/*.md
propose · apply · archive · new · continue · verify · explore 등 슬래시 커맨드.

.claude/skills/openspec-*/SKILL.md
각 명령에 대응하는 스킬. 슬래시 커맨드와 CLI가 같은 절차를 밟게 합니다.

재현: `git checkout step-00-init` — 초기화 직후 시점.

살아있는 교재: openspec-ecommerce-api, commit e94e129 chore: openspec init (--tools claude)

실습

Step 1~2 · Propose와 Specs

openspec new change로 change를 열면 폴더가 생기고 다음 명령을 안내합니다. instructions가 준 template을 채워 proposal.md를 쓰고, specs는 delta로 계약을 씁니다. #### Scenario는 정확히 hashtag 4개여야 합니다

openspec new change product-catalog

```
$ openspec new change "product-catalog"
- Creating change 'product-catalog' with schema
  'spec-driven'...
Created change 'product-catalog' at
  openspec/changes/product-catalog/
Schema: spec-driven
Next: openspec status --change product-catalog
```

wf-02-new-change · commit 5894739

함정 ① — specs의 `#### Scenario:`는 정확히 hashtag 4개여야 합니다. 3개(###)나 불릿(- Scenario)이면 delta 파서가 **조용히 무시**합니다. 작성 후 `openspec validate product-catalog --strict`로 통과를 확인합니다.

proposal.md — What Changes (실물) markdown

```
## What Changes

- 상품 자원 REST API 신설
  - POST /products 등록, GET /products 목록
  - GET /products/:id 단건 조회
  - PATCH /products/:id 부분 수정
  - DELETE /products/:id 삭제
- 상품 모델: id(문자열), name,
  price(정수 cents), stock(정수, 0 이상)
- 입력 검증: name 필수·비어있지 않음,
  price·stock은 정수이며 음수 불가 → 위반 시 400
- 미존재 상품 조회·수정·삭제 시 404
- 저장은 인메모리 리포지토리(교체 가능한
  인터페이스) 뒤에 둡니다
```

proposal-excerpt · commit 5894739

살아있는 교재: openspec-ecommerce-api, commit 5894739(proposal) · 769f95c(specs)

실습

Step 3~4 · Design과 Tasks

design은 데이터 모델·라우트 표·에러 모델과 함께 기각한 대안까지 남깁니다. tasks는 store → validation → routes(test-first) → wire-up의 TDD 순서 체크박스입니다. 여기서 status가 4/4 complete, apply ready를 표시합니다

design — 결정과 기각을 남긴다

요소	내용
데이터 모델	price는 정수 cents, id는 crypto.randomUUID()
라우트 표	메서드·경로·성공/실패 상태코드를 한 표로
에러 모델	{ "error": string } 통일
저장소 인터페이스	ProductStore — 인메모리 뒤에 교체 가능
기각한 대안	zod 검증(1막 과함), 클래스 도메인 모델(과설계)

"무엇을 안 했는가"를 남기는 것이 design의 가치입니다.

tasks — TDD 순서로 쪼갠다

1 **Store and model** 1.1 1.2
Product·ProductStore 인터페이스 정의, Map 기반 InMemoryProductStore 구현.

2 **Validation (test-first)** 2.1
validateProductInput의 실패 케이스 테스트를 먼저 쓰고 통과시킵니다.

3 **Routes → wire-up** 3 4
POST·GET·PATCH·DELETE 라우트를 만들고 index.ts에 마운트합니다.

openspec status → 4/4 complete, apply ready. 이 지점이 propose와 apply 사이의 사람 리뷰 포인트입니다 — 의도서에 서명하기 전에 읽는 자리입니다.

살아있는 교재: openspec-ecommerce-api, commit 8d4f6d3(design) · 2c25cd9(tasks)

실습

Step 5 · 무브 단위 구현

tasks 체크박스를 갱신하며 작은 커밋으로 쌓습니다. store → validation → 라우트(POST·GET, PATCH·DELETE) → 마운트 순서로, 무브 하나가 대략 커밋 하나 크기입니다. 각 무브는 실제 git 커밋으로 남습니다



함정 ② — PATCH의 body spread

요청 body를 그대로 spread하면 id가 오염됩니다. 실제 코드는 `pickProductFields(body)`로 허용 필드(name·price·stock)만 추출해 막습니다.

검증(400)을 존재 확인(404)보다 먼저

무효 입력이 store를 건드리지 못하게 순서를 고정합니다. 그래야 "400은 상품을 그대로 둔다" 시나리오가 성립합니다.

실측 git log step-00-init..act1-product-catalog (openspec-ecommerce-api)

실습

Step 6~7 · Verify와 Archive

구현이 spec과 일치하는지 3C(Completeness · Correctness · Coherence)로 점검하고, openspec archive product-catalog --yes로 delta를 본류 spec에 병합합니다

Verify — 3C 점검

1 Completeness

tasks 체크박스가 전부 [x]이고, 모든 spec requirement가 구현되었습니다.

tasks 완료 · requirement 전부

2 Correctness

각 scenario에 대응하는 테스트가 존재합니다. 이 change에서는 33개입니다.

33 tests · scenario별

3 Coherence

design 결정(저장소 인터페이스 · 에러 모델)이 코드에 그대로 반영되었습니다.

design 일치

bash — E2E smoke + archive

```
# 서버 기동 후 실제 계약을 밖에서 확인합니다
$ curl -s -o /dev/null -w "%{http_code}" -X POST /products -d @keyboard.json
201   상품 생성
$ curl -s -o /dev/null -w "%{http_code}" /products
200   목록 조회
$ curl -s -o /dev/null -w "%{http_code}" /products/does-not-exist
404   미존재 조회
$ curl -s -o /dev/null -w "%{http_code}" -X POST /products -d '{}'
400   무효 입력 거부

# 검증이 끝나면 계약을 본류 spec에 합칩니다
$ openspec archive product-catalog --yes
→ delta 5 requirements → openspec/specs/product-catalog/spec.md (+5 added)
→ change 폴더 이동: changes/archive/2026-07-08-product-catalog/
```

774e76c · tag act1-product-catalog

archive는 두 가지를 합니다. **delta 5개 requirement를 본류 spec.md로 병합**하고, **change 폴더를 archive로 이동**합니다. 이후 openspec list는 활성 변경 없음, openspec list --specs는 product-catalog(5 requirements)를 보여줍니다.

실측: openspec 1.5.0, 커밋 774e76c chore(product-catalog): archive, 태그 act1-product-catalog

예제

실제 spec — Create Product

본류로 병합된 spec.md 실물입니다. SHALL 하나에 성공·실패 시나리오를 함께 담습니다

spec.md markdown

```
### Requirement: Create Product
The system SHALL create a product from `name`, `price` (integer cents),
and `stock` (non-negative integer), assign a unique `id`, and return the
created product.

#### Scenario: Create with valid input
- **WHEN** a client sends `POST /products` with
  `{ "name": "Keyboard", "price": 4200, "stock": 10 }`
- **THEN** the system responds `201`
- **AND** the body contains the product with a generated `id` and the
  submitted `name`, `price`, `stock`

#### Scenario: Reject missing name
- **WHEN** a client sends `POST /products` without `name`, or with an empty `name`
- **THEN** the system responds `400`
- **AND** no product is created
```

specs/product-catalog/spec.md

SHALL

requirement 본문은 SHALL/MUST로 의무를 진술합니다. "할 수도 있다"가 아니라 "해야 한다"입니다.

성공과 실패를 한 requirement에

정상 생성 시나리오와 함께 Reject missing name · Reject negative price · Reject negative stock이 붙습니다. 실패 경로를 못박는 것이 계약의 핵심입니다.

Scenario

hashtag는 정확히 4개입니다. 이 형식만이 delta 파서가 시나리오로 인식하는 유일한 형태입니다.

관찰 가능한 동작

"store에 저장한다"가 아니라 "POST /products에 201로 응답하고 생성된 상품을 돌려준다"처럼 밖에서 관찰 가능한 계약으로 씁니다. 구현 세부는 design과 code의 몫입니다.

실측: openspec/specs/product-catalog/spec.md (태그 act1-product-catalog)

세 가지 함정

경고 없이 무시되거나 데이터가 오염되는 세 지점을, 형식·구현 규칙으로 막습니다

⚠️ WARNING **함정 ① · #### Scenario는 정확히 hashtag 4개**

Scenario는 hashtag가 정확히 4개여야 합니다. 3개(### Scenario)나 불릿(- Scenario)이면 delta 파서가 오류 없이 조용히 무시합니다. 통과했다고 착각하기 쉬워 가장 위험합니다.

관찰 불가 → validate --strict 통과처럼 보여도 시나리오가 계약에서 빠집니다. (delta 규칙은 심화 LO에서 깊이 다룹니다.)

🚫 ERROR **함정 ② · PATCH의 id 오염**

PATCH에서 요청 body를 그대로 spread({ ...existing, ...body })하면 클라이언트가 보낸 id가 기존 id를 덮어써 오염됩니다.

방어: pickProductFields(body)로 허용 필드(name · price · stock)만 추출해 반영합니다. id 같은 불변 필드는 애초에 받지 않습니다.

💡 TIP **함정 ③ · 검증(400)을 존재 확인(404)보다 먼저**

무효 입력이 store를 건드리지 못하게 검증(400)을 존재 조회(404)보다 앞에 둡니다. 순서를 고정해야 부작용 없는 거부가 보장됩니다.

그래야 "400은 상품을 그대로 둔다(product left unchanged)" 시나리오가 성립합니다.

설명

임의 시점 재현

git checkout으로 change 완료 시점도, spec 직후 같은 중간 단계도 되살립니다

bash — 임의 시점 재현

```
# change 1(상품 카탈로그) 완료 시점 전체
$ git checkout act1-product-catalog
HEAD → 774e76c  (archive 완료, 본류 spec 5 requirements)

# spec 작성 직후(구현 전) — 계약만 있고 코드는 없는 상태
$ git checkout 769f95c
detached HEAD  spec for product CRUD

# store 무브 시점 — 인메모리 리포지토리만 있는 상태
$ git checkout a03ed2c
detached HEAD  add product store (in-memory)
```

tags: step-00-init · act1-product-catalog

태그는 **완료된 막(act)**을, 커밋 해시는 **막 안의 중간 단계**를 가리킵니다. 강의 어느 지점에서든 그 상태로 이동해 실습을 이어갈 수 있습니다.

제공 태그

- `step-00-init` openspec 초기화 직후
- `act1-product-catalog` 상품 CRUD change 완료
- `act1-order-management` 주문 관리 change 완료
- `act2-order-stock-and-cancellation` 재고·취소 change 완료

실측 git log: openspec-ecommerce-api, 태그 step-00-init · act1-* · act2-*

레퍼런스

OpenSpec 구현체 · AI-DLC 상위 방법론(AWS) · 버전과 설치 · 선행 LO의 정확한 출처입니다

구현체 · Fission-AI

OpenSpec

SDD의 산출물 지도를 파일과 CLI로 실현한 도구입니다. 핵심 개념은 spec delta이며 30여 개 AI 도구를 지원합니다.

예시: github.com/Fission-AI/OpenSpec · openspec.dev

방법론 · AWS

AI-DLC

OpenSpec이 구현하는 상위 방법론입니다. 슬로건은 "AI proposes, human approves", 단위는 Bolts와 Mob Elaboration/Construction입니다.

예시: aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/ · re:Invent 2025 DVT214

환경

버전 · 설치

npm @fission-ai/openspec v1.5.0(2026-07-08 확인), Node 20.19 이상이 필요합니다.

예시: npm i -g @fission-ai/openspec@latest

SDD 원리

선행 LO

SDD의 원리와 산출물 지도를 세운 LO입니다. "의도가 진실의 원천"이라는 원리 위에 OpenSpec이 놓입니다.

예시: LO-SDD 산출물 지도 (Frentis 2026)

LO5 · OPENSPEC 기초 ARTIFACT-GUIDED 사이클

체크포인트

스스로 답할 수 있으면 LO5를 통과한 것입니다. 초기화 산출물부터 git 태그 재현까지 일곱 가지를 점검합니다

01 init 산출물 두 가지

openspec init --tools claude --force가 만드는 것: openspec/config.yaml과 .claude/commands/opsx/*, skills.

02 artifact DAG 순서

proposal 다음 (specs, design), 그다음 tasks. applyRequires는 tasks — 부모 없이 자식을 못 만듭니다.

03 status vs instructions

status는 완료 판정과 DAG를, instructions는 산출물의 template과 제약을 줍니다. 역할이 다릅니다.

04 context · rules 파일 금지

두뇌를 위한 제약이지 산출물이 아닙니다. 파일에 복붙하면 스펙이 config를 중복해 오염됩니다.

05 #### Scenario는 4개

hashtag가 정확히 4개여야 합니다. 3개나 불릿이면 delta 파서가 조용히 무시합니다.

06 archive의 두 가지

delta를 본류 spec.md에 병합하고, change 폴더를 archive로 이동합니다.

07 git 태그 재현

git checkout act1-product-catalog로 change 1 완료 시점을, 커밋 해시로 중간 단계를 되살립니다.

✅ 도구가 절차를 주도하고, 사람은 의도를 검토합니다. 이것이 artifact-guided 사이클의 핵심입니다.

SECTION

06 OpenSpec 심화 — delta 변경 관리

- delta의 세 형식(ADDED · MODIFIED · REMOVED)을 구분하고 archive 병합 결과를 읽습니다
- 기존 동작을 바꿀 때 MODIFIED delta로 requirement 블록을 전체 복사해 부분 손실 함정을 피합니다
- 한 change에 MODIFIED와 ADDED가 공존하는 변경(주문 재고 차감 + 취소)을 직접 설계·구현합니다
- verify 3C와 확장 워크플로우(new · continue · sync · bulk-archive · explore)를 언제 쓸지 판단합니다

50분 (블록 6)

개념

변경분으로서의 delta

기초는 새 기능만 쌓았지만, 실무는 대부분 이미 있는 동작을 바꾸는 일입니다

delta — change 폴더의 spec

`openspec/changes/<name>/specs/…`

이번 변경분만 담습니다. 헤더(ADDED / MODIFIED / REMOVED)가 변경 종류를 정합니다.

living spec — 본류

`openspec/specs/<capability>/spec.md`

현재 전체 계약입니다. archive가 delta를 이 본류에 병합해 진실을 하나로 유지합니다.

여기서 OpenSpec의 진짜 값이 드러납니다

ADDED는 새로 더하고, MODIFIED는 기존 requirement를 바꾸고, REMOVED는 없앱니다. archive가 이 델타를 본류에 병합합니다. 이 병합 규칙 때문에 MODIFIED에는 함정이 하나 있습니다.

이 LO는 change 3(order-stock-and-cancellation)으로 그 함정을 실물로 밟습니다.

설명

delta의 세 형식

delta의 헤더가 변경 종류를 정합니다. 한 delta 파일 안에 이 헤더들이 공존할 수 있습니다. change 3(order-stock-and-cancellation)은 ## MODIFIED Requirements(Create Order 변경)와 ## ADDED Requirements(Cancel Order 신설)를 한 파일에 함께 담았습니다

`## ADDED Requirements`	새 requirement를 더합니다.	archive 시 **본류에 추가**	`+N added`
`## MODIFIED Requirements`	기존 requirement를 이름으로 찾아 통째로 교체합니다.	archive 시 **본류의 같은 이름을 대체**	`~N modified`
`## REMOVED Requirements`	requirement를 삭제합니다.	archive 시 **본류에서 제거**	`-N removed`

한 delta 파일에 세 헤더가 **공존**할 수 있습니다. change 3은 MODIFIED(Create Order)와 ADDED(Cancel Order)를 한 파일에 함께 담아 실증했습니다.

실측: 태그 act2-order-stock-and-cancellation, commit 626919c(delta) · 8620f87(archive)

심화

MODIFIED의 함정 — 전체 복사

MODIFIED는 이름으로 본류 requirement를 통째로 갈아끼웁니다. 그래서 전체를 복사해야 합니다

🚫 ERROR **이름이 곧 교체 키**

archive는 delta의 requirement 이름으로 본류의 같은 이름 블록을 찾아 통째로 대체합니다. 부분 병합이 아닙니다.

⚠️ WARNING **부분만 쓰면 나머지 소실**

MODIFIED에 시나리오를 하나만 적으면, archive 후 본류에는 그 하나만 남고 나머지가 사라집니다.

"partial content loses detail at archive time" — 조용히 일어나므로 발견이 늦습니다.

💡 TIP **방어는 전체 복사 하나**

기존 requirement 블록(모든 시나리오 포함)을 전부 복사한 뒤, 필요한 부분만 고치고 새 시나리오를 더합니다.

change 3의 Create Order

하나만 새로 넣는 변경이지만, delta에는 **다섯 개**가 모두 실립니다.

기존	`valid items`
기존	`empty items`
기존	`non-positive quantity`
기존	`unknown product`
신규	`insufficient stock (409)`

LO 원문 · docs/steps/03-order-stock-and-cancellation.md (partial content loses detail at archive time)

실습

Step 1 · Propose

change 2 완료 시점에서 시작합니다. 이때 Create Order는 재고 개념이 없습니다

bash — 출발점 확인 후 change 제안

```
# change 2 완료 시점 — 재고 개념 없음
$ git checkout act1-order-management
$ openspec list --specs
product-catalog
order-management
# Create Order에 재고 시나리오 없음 확인
$ openspec show order-management --json | less

$ openspec new change "order-stock-and-cancellation"
Created change 'order-stock-and-cancellation'
Next: openspec status --change order-stock-…
```

commit 2d811f0

proposal.md — Capabilities (실물) markdown

```
## What Changes

- 주문 생성 시 각 상품의 재고를 확인 →
  부족하면 409, 충분하면 주문 수량만큼
  stock 차감 (BREAKING: 기존 Create Order
  동작이 바뀝니다)
- 주문 취소: POST /orders/:id/cancel →
  status를 "cancelled"로, 차감 재고 복원

## Capabilities

### New Capabilities
(없음)

### Modified Capabilities
- order-management: Create Order에 재고
  차감·부족 시 409를 추가하고, Cancel Order
  요구사항을 신설합니다
```

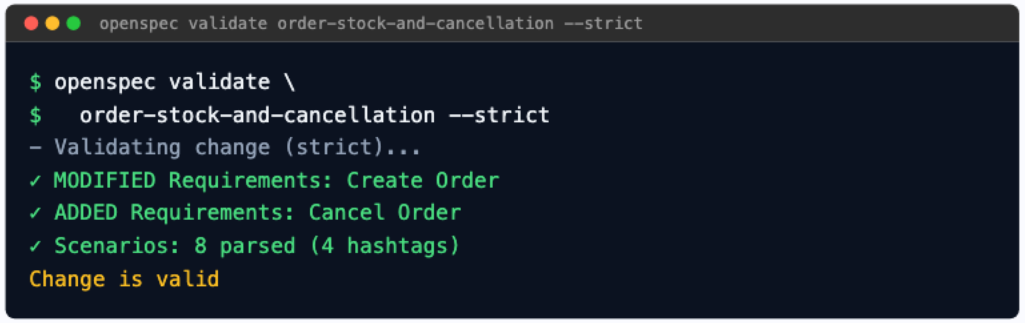
proposal.md · commit 2d811f0

실측: 태그 act1-order-management → commit 2d811f0(propose) · proposal.md

실습

Step 2 · Specs 검증

delta 형식 오류는 경고 없이 걸러집니다. #### Scenario는 hashtag 4개가 아니면 무시됩니다



strict gate · commit 626919c

인식되는 유일한 형태

`#### Scenario: Reject insufficient stock`

hashtag 정확히 4개. delta 파서가 시나리오로 인식하는 형태입니다.

조용히 무시되는 형태

`### Scenario: …` (hashtag 3개)

`- Scenario: …` (불릿)

에러 없이 그냥 사라집니다. validate --strict가 잡습니다.

실측: openspec 1.5.0, validate order-stock-and-cancellation --strict

예제

MODIFIED delta 실물 — Create Order

MODIFIED 헤더 아래 Create Order입니다. requirement 문장 자체에 재고 차감이 더해집니다

specs/order-management/spec.md (delta) markdown

```markdown
## MODIFIED Requirements

### Requirement: Create Order
The system SHALL create an order from a non-empty
list of items — each referencing an existing product
`id` with a `quantity` of at least 1 and sufficient
available stock — ... decrement each product's
`stock` by the ordered quantity, set `status` to
`"pending"`, and return the created order.

#### Scenario: Create order with valid items ...
#### Scenario: Reject empty items ...
#### Scenario: Reject non-positive quantity ...
#### Scenario: Reject unknown product ...

#### Scenario: Reject insufficient stock
- **WHEN** a client sends `POST /orders` for a
  product whose available `stock` is less than the
  requested `quantity`
- **THEN** the system responds `409`
- **AND** no order is created
- **AND** no product's stock is changed
```

MODIFIED Requirements

본문에 재고가 들어감

requirement 문장 자체에 sufficient available stock과 decrement stock이 추가됐습니다. change 2 원문에는 없던 계약입니다.

기존 4개 + 신규 1개

valid / empty / quantity / unknown을 그대로 옮기고 Reject insufficient stock을 더합니다. 하나만 새로 넣는 변경이지만 delta에는 다섯 개가 다 실립니다. 이것이 전체 복사 규칙입니다.

409의 삼중 보장

insufficient stock은 409 + no order is created + no product's stock is changed 세 AND를 못박습니다. 부분 차감이 일어나면 세 번째 AND가 깨집니다.

실측: changes/archive/2026-07-08-order-stock-and-cancellation/specs/order-management/spec.md · commit 626919c

예제

ADDED delta 실물 — Cancel Order

ADDED 헤더 아래 Cancel Order입니다. 상태 전이와 재고 복원을 한 묶음으로 처리합니다

specs/order-management/spec.md (delta) markdown

```markdown
## ADDED Requirements

### Requirement: Cancel Order
The system SHALL cancel a pending order by `id`, set
its `status` to `"cancelled"`, and restore the stock
the order had consumed. Cancelling an order that is
already cancelled SHALL fail.

#### Scenario: Cancel a pending order
- **WHEN** a client sends `POST /orders/{id}/cancel`
  for a pending order
- **THEN** the system responds `200`
- **AND** the order `status` becomes `"cancelled"`
- **AND** each product's `stock` is increased back by
  that order's quantities

#### Scenario: Cancel a missing order
- **WHEN** … for an `id` that does not exist
- **THEN** the system responds `404`

#### Scenario: Reject cancelling an already-cancelled order
- **WHEN** … for an order whose `status` is already
  `"cancelled"`
- **THEN** the system responds `409`
- **AND** no product's stock is changed
```

ADDED Requirements

상태 전이

pending → cancelled. 취소는 이 전이와 재고 역연산(복원)을 한 묶음으로 진행합니다.

재취소 방어

이미 cancelled면 409 + 재고 불변. 역연산을 두 번 걸어 재고가 부풀지 않게 막는 계약입니다. 구현은 상태를 먼저 확인해 되돌립니다.

왜 ADDED인가

Cancel Order는 본류에 없던 새 requirement입니다. 그래서 MODIFIED가 아니라 ADDED로, 같은 delta 파일 안에 함께 둡니다.

실측: changes/archive/2026-07-08-order-stock-and-cancellation/specs/order-management/spec.md · commit 626919c

설명

Step 3~4 · Design과 Tasks

핵심 결정은 재고 원자성입니다. 전부 확인한 뒤, 하나라도 부족하면 아무것도 바꾸지 않습니다

design 결정 — 검증 순서 (src/orders.ts 주석)

```
# 검증 순서: 뒤로 갈수록 통과 뒤에만 실행
(1) 구조 검증          → 400
(2) 상품 참조 검증     → 404
(3) 재고 확인(전량)    → 409
(4) 차감·저장

# 취소 = 역연산
cancelled로 상태 변경 + stock 복원
이미 cancelled면 → 409
```

check-all-then-deduct · commit 7b86f21

tasks 분해 순서

1. 재고 확인 로직 (aggregate로 productId별 합산)
2. 전량 통과 후 일괄 차감
3. 취소 = 역연산 (상태 전이 + 재고 복원)
4. 시나리오별 테스트 (201·400·404·409)

원자성이 핵심 — 부분 차감을 허용하면 "no product's stock is changed" 계약이 깨집니다. 그래서 확인과 차감을 두 단계로 분리합니다.

실측: commit 7b86f21(design) · 472c7ca(tasks) · src/orders.ts 검증 순서 주석

예제

Step 5 · 원자적 재고 차감

aggregate로 productId별 합산 수량을 구해 전량을 먼저 확인한 뒤 일괄 차감합니다. 취소는 상태 전이와 재고 복원의 역연산입니다

src/orders.ts — typescript

```typescript
// productId별 합산 수량(중복 item 대응)
const required = aggregate(parsed)

// 3) 재고 확인(전량) — 하나라도 부족하면
//    아무것도 바꾸지 않는다
for (const [productId, qty] of required) {
  const product = products.get(productId)!
  if (product.stock < qty) {
    return c.json(
      { error: `insufficient stock for ${productId}` },
      409)
  }
}

// 4) 전량 통과 후에만 차감
for (const [productId, qty] of required) {
  const product = products.get(productId)!
  products.update(productId,
    { stock: product.stock - qty })
}

// 취소 = 역연산
routes.post('/orders/:id/cancel', (c) => {
  if (order.status === 'cancelled') {
    return c.json({ error: 'order already cancelled' },
      409)
  }
  for (const [pid, qty] of aggregate(order.items)) {
    // stock 복원: stock + qty
  }
  orders.setStatus(order.id, 'cancelled')
})
```

aggregate · check-all-then-deduct · cancel

aggregate로 중복 합산

같은 productId를 두 줄로 주문해도 합쳐서 재고를 봅니다. 줄 단위로 보면 각각은 통과해도 합이 재고를 초과할 수 있습니다.

확인과 차감을 분리

확인 루프를 전부 돈 뒤에야 차감 루프를 시작합니다. 중간에 409면 어떤 stock도 바뀌지 않습니다.

취소는 대칭 역연산

차감했던 수량을 그대로 복원합니다. 이미 cancelled면 먼저 409로 막아 이중 복원을 방지합니다.

실측: src/orders.ts (태그 act2-order-stock-and-cancellation, commit aad296b)

실습

Step 6 · 본류 병합

archive가 delta를 본류에 병합하고 change 폴더를 옮깁니다. +1 added는 Cancel 추가, ~1 modified는 Create 교체입니다

openspec archive order-stock-and-cancellation --yes

```
$ openspec archive \
$   order-stock-and-cancellation --yes
Applying changes to openspec/specs/
  order-management/spec.md:
  + 1 added      # Cancel Order
  ~ 1 modified   # Create Order (재고 반영)

Archived to changes/archive/
  2026-07-08-order-stock-and-cancellation/
```

병합 로그 실측 · commit 8620f87

order-management의 Create Order

change 2 직후 (본류)
시나리오 4개, 재고·409 없음

change 3 delta (MODIFIED)
시나리오 5개, 본문에 재고 문구

change 3 archive 후 (본류)
delta로 교체됨 → 5개 + Cancel Order 추가

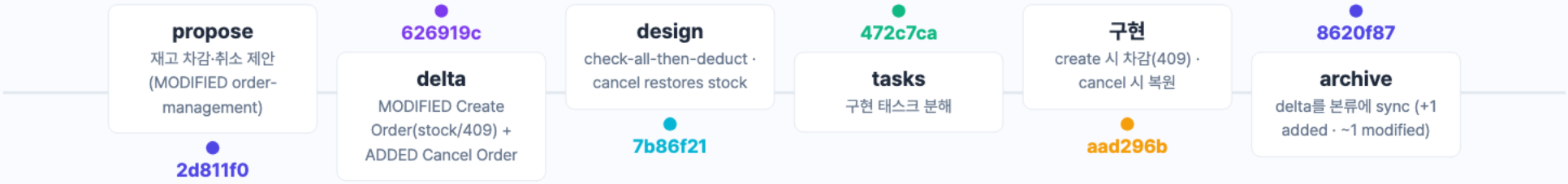
MODIFIED에 4개만 적었다면 archive 후 본류도 4개로 줄고, 새로 넣으려던 재고 시나리오만 남았을 것입니다. 그래서 전체 복사가 규칙입니다.

실측: openspec archive order-stock-and-cancellation --yes · commit 8620f87

실습

구현 무브 재현

change 3의 전 과정이 git 커밋으로 남아, 각 단계를 그대로 되살립니다



중간에 별도 커밋 두 개 더

98c0e6d에서 Order status union과 setStatus를 store에 먼저 넣습니다. 상태 전이의 토대를 구현 전에 마련하는 순서입니다.

무브 하나가 커밋 하나

artifact(delta·design·tasks)와 구현이 각각 독립 커밋입니다. 어느 단계로든 checkout해 그 시점의 spec과 코드를 재현할 수 있습니다.

실측 git log (openspec-ecommerce-api, 태그 act1-order-management..act2-order-stock-and-cancellation)

심화

확장 워크플로우

기본 사이클(propose → apply → verify → archive)만으로 대부분을 처리하지만, 규모가 커지면 확장 명령이 붙습니다. sync는 archive 없이 델타만 본류에 점진 반영하고, bulk-archive는 여러 change를 일괄 병합하며 충돌을 코드 기준으로 해결합니다

`/opsx:new <name>`	artifact를 한 번에 만들지 않고 하나씩 상의하며
`/opsx:continue [name]`	new 이후 다음 artifact를 이어서 하나씩 생성
`/opsx:ff <name>`	propose와 동일 — 네 artifact 빠른 일괄 생성
`/opsx:explore [topic]`	구현 전 문제 공간 탐색(코드 없음)
`/opsx:verify [name]`	구현이 spec과 맞는지 3C 검증
`/opsx:sync [name]`	archive 없이 delta를 본류에 병합(활성 유지)
`/opsx:bulk-archive`	여러 change 일괄 아카이브 + 충돌 자동 해결

선택 가이드

단순 · 빠름
`propose(ff) → apply → archive`

복잡 · 팀 검토
`new → continue(반복) → apply → verify → archive`

병렬 다기능
`기능별 new/ff → 각자 apply → sync로 점진 반영 → bulk-archive`

아이디어 단계
`explore 먼저`

레퍼런스

OpenSpec 구현체 · spec delta 개념 · 살아있는 교재(repo·태그·커밋) · 선행 LO의 정확한 출처입니다

구현체 · Fission-AI

OpenSpec

SDD의 산출물 지도를 파일과 CLI로 실현한 도구입니다. 핵심은 spec delta로, change마다 requirement 변화를 캡처하고 archive가 본류(living spec)에 병합합니다.

예시: github.com/Fission-AI/OpenSpec · openspec.dev · npm @fission-ai/openspec v1.5.0 (Node 20.19+)

핵심 개념

spec delta

ADDED / MODIFIED / REMOVED 헤더로 변경분을 기술합니다. MODIFIED는 이름으로 본류 requirement를 통째로 교체하므로 전체 복사가 규칙입니다.

예시: #### Scenario(hashtag 4개) · openspec validate <change> --strict

관통 예제 · repo

살아있는 교재

change 3(order-stock-and-cancellation)의 실제 delta·구현·커밋으로 이 LO를 관통합니다. 태그로 임의 단계를 재현합니다.

예시: openspec-ecommerce-api · commit 626919c(delta)·8620f87(archive) · 태그 act2-order-stock-and-cancellation

OpenSpec 기초

선행 LO

artifact-guided 사이클(propose → apply → archive)과 ADDED delta를 다룬 LO입니다. 이 심화 LO는 그 위에 MODIFIED와 확장 워크플로우를 얹습니다.

예시: LO-OpenSpec 기초 artifact-guided 사이클 (Frentis 2026)

LO6 · OPENSPEC 심화: DELTA 변경 관리

체크포인트

스스로 답할 수 있으면 LO6을 통과한 것입니다. delta 세 형식부터 확장 워크플로우까지 여섯 가지를 점검합니다

01 delta 세 헤더

ADDED(+N added) · MODIFIED(~N modified) · REMOVED. 한 delta 파일에 공존할 수 있습니다.

02 MODIFIED 전체 복사

archive는 이름으로 requirement를 통째 교체합니다. 부분만 쓰면 나머지 시나리오가 소실됩니다.

03 #### Scenario 4개

hashtag가 정확히 4개여야 파서가 인식합니다. validate --strict가 형식 위반을 잡습니다.

04 check-all-then-deduct

전량 확인 후에만 차감합니다. 부분 차감과 "no stock changed" 위반을 막는 원자성입니다.

05 ~1 modified, +1 added

archive 로그 해석: Create Order 교체 + Cancel Order 추가. 병합 결과를 읽습니다.

06 sync vs bulk-archive

sync는 change를 닫지 않고 본류 반영. bulk-archive는 코드베이스를 조사해 실제 구현된 것만 병합.

✅ ADDED로 쌓는 데서 MODIFIED로 진화시키는 데로. 여기서 OpenSpec의 진짜 값이 드러납니다.

SECTION

07 Spec Kit 기초 워크플로우

- GitHub Spec Kit(specify)을 uv tool install로 설치하고 specify init . --integration claude로 초기화합니다
- constitution → specify → plan → tasks → implement 다섯 단계가 무엇을 만들고 무엇을 검증하는지 익힙니다
- create-new-feature.sh --json이 번호 폴더를 스캐폴드하고 내용은 LLM 스킬이 채우는 분업을 직접 확인합니다
- spec의 User Story(P1/P2/P3)·FR·Success Criteria를 읽고 git 태그 speckit-product-catalog로 재현합니다

50분 (블록 7)

개념

Spec Kit 개요

코드를 짜기 전에 무엇을·왜를 명세로 확정하는 GitHub의 SDD 도구입니다

ℹ️ INFO **무엇을 만드는가**

Spec Kit은 스펙 주도 개발(SDD)을 Claude Code 같은 코딩 에이전트 위에서 구조화합니다. 각 단계가 Markdown artifact를 남기고, 에이전트는 그 산출물을 근거로 다음 단계를 진행합니다.

CLI 이름 specify · 이 교재 기준 0.12.8(실측 0.12.8.dev0) · 30여 개 AI 에이전트 지원

💡 TIP **OpenSpec과의 갈림길**

OpenSpec은 하나의 살아있는 명세에 델타(추가·수정·삭제)를 얹어 진화시킵니다. Spec Kit은 기능(feature)마다 번호 붙은 폴더를 새로 만들고, 그 안에 완결된 명세·설계·작업 문서를 upfront로 갖춥니다.

변경이 필요하면 델타 대신 새 feature 폴더를 만듭니다. 이 차이는 심화(LO8·LO9)에서 정면으로 다룹니다.

⚠️ WARNING **spec은 델타가 아니라 전체 명세**

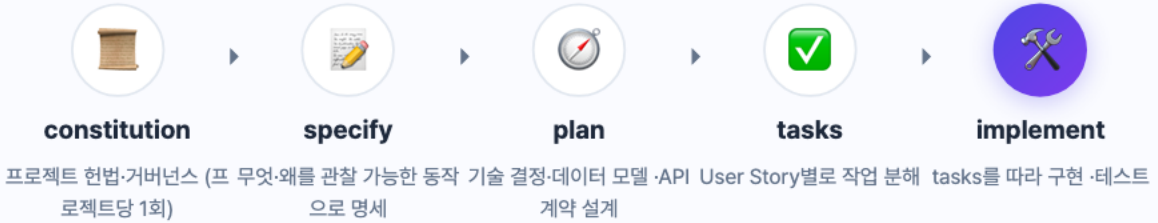
OpenSpec의 ADDED/MODIFIED/REMOVED 델타와 달리, Spec Kit의 spec.md는 그 기능의 완전한 관찰 가능 동작(상태 코드·페이로드·오류 조건)을 서술합니다.

관통 예제는 상품 카탈로그 CRUD(feature 001-product-catalog)이며 살아있는 교재는 speckit-ecommerce-api입니다.

설명

다섯 단계 워크플로우

constitution(헌법) → specify(무엇·왜) → plan(어떻게) → tasks(작업 분해) → implement(구현). 각 단계는 /speckit.* 슬래시 커맨드로 실행하며, 앞 단계의 산출물을 근거로 다음 단계를 진행합니다



🚦 사이사이 선택 품질 게이트: specify 뒤 clarify · plan 뒤 checklist · tasks 뒤 analyze. constitution은 1회, specify 이후는 feature마다 반복합니다.

설명

단계별 산출물 대응

각 단계는 슬래시 커맨드로 실행해 정해진 위치에 Markdown을 남기고, 뒤 단계가 앞 산출물을 받습니다

단계	슬래시 커맨드	하는 일	산출물
헌법	/speckit.constitution	프로젝트 전반의 원칙·거버넌스를 정한다	.specify/memory/constitution.md
명세	/speckit.specify	무엇·왜를 관찰 가능한 동작으로 쓴다	specs/NNN-name/spec.md
계획	/speckit.plan	기술 결정·데이터 모델·API 계약을 설계한다	plan.md + research·data-model·contracts·quickstart
작업	/speckit.tasks	User Story별로 작업을 분해한다	tasks.md
구현	/speckit.implement	tasks를 따라 코드를 짜고 테스트한다	src/ + 테스트

실습

설치와 초기화

specify는 Python 기반이라 uv tool install로 전역 설치합니다. specify --version으로 0.12.8을 확인하고, specify init . --integration claude로 현재 디렉터리를 Claude Code 연동으로 초기화합니다

bash

```bash
# specify CLI 전역 설치 (uv tool, git+https)
$ uv tool install specify-cli \
$   --from git+https://github.com/github/spec-kit.git
# 버전 확인 (2026-07-08 실측)
$ specify --version
specify 0.12.8.dev0
# 현재 디렉터리에 초기화 (Claude Code 연동)
$ specify init . --integration claude
✔ Initialized Spec Kit (integration: claude)
```

spk-version.txt · commit 4cbb344

--integration claude — Claude Code용 연동 파일과 /speckit.* 슬래시 커맨드를 함께 설치합니다. **.** 은 현재 디렉터리에 초기화한다는 뜻입니다.

초기화가 남기는 확인 명령

초기화 결과 확인

```
$ ls .specify/
templates  scripts  memory  workflows
feature.json  init-options.json  integration.json
$ ls .specify/scripts/bash/
create-new-feature.sh  setup-plan.sh
setup-tasks.sh  check-prerequisites.sh  common.sh
$ ls .claude/skills/
speckit-constitution  speckit-specify ...
speckit-plan  speckit-tasks  speckit-implement
$ cat .specify/init-options.json
{ "ai":"claude", "integration":"claude",
  "script":"sh", "feature_numbering":"sequential" }
```

.specify/ 실측 · commit 4cbb344

살아있는 교재: speckit-ecommerce-api, commit 4cbb344 chore: specify init (--integration claude)

설명

초기화 산출물

.specify/는 도구가 쓰는 재료(templates·scripts·memory·workflows), .claude/skills/speckit-*는 사용자가 부르는 명령입니다. CLI 스크립트는 폴더·경로만 계산하고, 명세·계획·작업의 실제 내용은 스킬(LLM)이 채웁니다

`.specify/` — 도구가 쓰는 재료

경로	역할
templates/	spec·plan·tasks·constitution·checklist 템플릿. 스크립트가 새 폴더로 복사
scripts/bash/	create-new-feature·setup-plan·setup-tasks·check-prerequisites·common
memory/constitution.md	프로젝트 헌법
workflows/speckit/	specify→plan→tasks→implement를 리뷰 게이트와 함께 정의한 workflow.yml

`.claude/skills/speckit-*` — 부르는 명령

스킬	슬래시 커맨드로 하는 일
speckit-constitution	헌법 작성 — /speckit.constitution
speckit-specify / clarify	명세 작성 · 모호점 질문·해소
speckit-plan / checklist	설계 산출물 작성 · 요구사항 품질 점검
speckit-tasks / analyze	작업 분해 · 정합성 분석
speckit-implement	tasks 따라 구현 — /speckit.implement

핵심 분업 — 스킬 이름은 `speckit-constitution`처럼 하이픈이고, 슬래시 커맨드는 `/speckit.constitution`처럼 점입니다. bash 스크립트는 **폴더·파일 스캐폴드와 경로 계산만** 하고, 명세·계획·작업의 **실제 내용은 스킬(LLM 주도)이 채웁니다.**

살아있는 교재: speckit-ecommerce-api, .specify/ · .claude/skills/ 실측 (commit 4cbb344)

예제

Step 1 · constitution (헌법)

/speckit.constitution이 .specify/memory/constitution.md에 프로젝트 원칙을 씁니다. 이 예제는 5개 Core Principle을 두며, 이후 plan/tasks 앞의 거버넌스 게이트로 작동합니다

Claude Code — 슬래시 커맨드

```
$ /speckit.constitution
→ speckit-constitution 스킬 실행
  write .specify/memory/constitution.md
Ratified v1.0.0 — 5 Core Principles
```

commit 0b215bf

헌법에는 5원칙 외에 **기술 제약**(TypeScript·Hono·pnpm·vitest·tsx)과 **개발 워크플로우**(specify → plan → tasks → implement, Constitution Check 게이트)도 함께 적습니다.

Version 1.0.0 · Ratified 2026-07-08 · 변경 시 버전 상향

constitution.md — 5개 Core Principle (실물)

원칙	요지
I. Spec-Driven (NON-NEGOTIABLE)	모든 기능은 코드 전에 spec(무엇·왜)으로 시작한다
II. Test-First	각 FR은 최소 하나의 테스트로 매핑. 실패 경로(400·404·409)도 명세·테스트
III. Thin, Swappable Storage	저장소는 인터페이스 뒤(지금은 인메모리). 라우트는 구체 저장소에 의존 금지
IV. Explicit Errors	모든 오류는 적절한 HTTP 상태 + { error } 본문. 예상 조건에 500 금지
V. Server Authority	서버가 파생 값(총액·재고 변화)을 계산. 클라이언트가 보낸 파생 필드는 무시

살아있는 교재: speckit-ecommerce-api, commit 0b215bf docs(constitution): ratify v1.0.0

실습

Step 2 · specify

/speckit.specify 안에서 create-new-feature.sh가 실행되어 번호 붙은 feature 폴더와 spec 템플릿을 스캐폴드합니다. --json으로 스크립트가 계산한 BRANCH_NAME·SPEC_FILE·FEATURE_NUM을 직접 확인할 수 있습니다

create-new-feature.sh --json (실측)

```
# /speckit.specify가 내부적으로 호출하는 스크립트
$ .specify/scripts/bash/create-new-feature.sh --json \
$   "Product catalog REST API: create, list, get,
$    update, delete products ..."
# To persist:
#  export SPECIFY_FEATURE=001-product-catalog-crud
#  export SPECIFY_FEATURE_DIRECTORY=.../specs/001-...
{ "BRANCH_NAME":"001-product-catalog-crud",
  "SPEC_FILE":".../specs/001-product-
           catalog-crud/spec.md",
  "FEATURE_NUM":"001" }
```

spk-create-feature.txt · specify 0.12.8.dev0

FEATURE_NUM 자동 채번

specs/ 아래 기존 폴더에서 가장 큰 번호를 찾아 +1 하고 3자리 zero-pad(001). --number로 수동 지정도 가능합니다.

BRANCH_NAME · 폴더 생성

설명에서 불용어를 걸러 짧은 이름을 만들고 번호를 붙여 specs/001-.../ 폴더와 브랜치 이름을 계산합니다. --short-name으로 이름을 다듬을 수 있어, 이 교재 저장소는 001-product-catalog로 커밋했습니다.

spec 템플릿 복사

spec-template.md를 새 폴더의 spec.md로 복사합니다. 여기까지가 스크립트의 일이고, 자리표시자는 LLM(speckit-specify 스킬)이 채웁니다.

핵심: 스크립트는 **번호·폴더·경로만** 만들고, spec의 내용은 채우지 않습니다.

실측: create-new-feature.sh --json (specify 0.12.8.dev0) · spk-create-feature.txt

예제

spec.md의 User Story

User Story는 누가·무엇을·왜를 평문으로 쓰고, 우선순위(P1/P2/P3)와 독립 테스트 방법을 명시합니다. 각 스토리는 독립적으로 테스트 가능한 MVP 슬라이스여야 하며, P1만 구현해도 쓸 만한 최소 제품이 나옵니다

spec.md — User Story 1 (실물) markdown

```
### User Story 1 – Register and browse
    products (Priority: P1)

카탈로그 운영자가 판매할 상품을 등록하고,
전체 목록과 개별 상품을 조회한다.

**Why this priority**: 상품이 없으면 주문도
판매도 불가능하다. 등록·조회는 카탈로그의
최소 기능이며, 이것만으로도 가치를 준다.

**Independent Test**: 상품을 하나 등록한 뒤
목록·단건 조회로 그 상품이 보이는지 확인.

**Acceptance Scenarios**:
1. Given 빈 카탈로그, When 상품을 등록하면,
   Then 식별자와 함께 반환된다 (201).
4. Given 존재하지 않는 식별자, When 조회하면,
   Then 찾을 수 없음 (404).
```

spec.md · P1 · commit e2cc35a

우선순위 = MVP 슬라이스

P1(등록·조회) / P2(수정) / P3(삭제). 각 스토리는 독립적으로 테스트 가능하며, P1만으로도 최소 제품이 됩니다. "Why this priority"가 그 순서의 근거입니다.

Independent Test

이 스토리만 떼어 검증하는 방법을 한 줄로 적습니다. 다른 스토리에 의존하지 않아야 병렬·점진 개발이 가능합니다.

Given / When / Then

수용 기준을 상태→행위→기대 결과로 씁니다. 성공(201)뿐 아니라 실패(404·400)도 시나리오로 적고, 이것이 그대로 vitest 테스트로 옮겨집니다.

살아있는 교재: speckit-ecommerce-api, specs/001-product-catalog/spec.md (commit e2cc35a)

예제

spec.md의 요구사항·기준

FR은 시스템이 반드시 해야 하는(MUST) 동작을 번호로 나열하고, 관찰 가능한 동작(상태 코드·조건)으로만 씁니다. SC는 기술 중립적이고 측정 가능한 성공 지표로, 무엇이 검증되면 이 기능이 완성인지를 정의합니다

Functional Requirements (실물) — markdown

```
- FR-001: MUST create a product from a name,
  a price in integer cents, and a non-negative
  integer stock, assign an id, return it (201).
- FR-002: MUST reject creation when name is
  missing/empty or price/stock is negative or
  non-integer (400), and create nothing.
- FR-005: Users MUST partially update name,
  price, or stock under the same validation
  (200); missing product 404; an invalid
  update (400) leaves the product unchanged.
- FR-006: Users MUST delete a product (204),
  or 404 if it does not exist.
```

spec.md · FR-001~006 · e2cc35a

Success Criteria (실물) — markdown

```
- SC-001: All five operations (create, list,
  get, update, delete) are exercisable
  end-to-end against a running server.
- SC-002: Every invalid path 400, every
  missing path 404; no expected condition 500.
- SC-004: An invalid update never mutates
  the stored product.
```

spec.md · SC-001~004 · e2cc35a

해설 — FR은 구현을 담지 않습니다. FR-002의 **"create nothing"**, FR-005의 **"leaves the product unchanged"**처럼 실패 시 부작용 없음까지 명시하는 것이 핵심입니다. SC는 기술 중립 지표라 어떤 구현이든 같은 기준으로 검증합니다.

살아있는 교재: speckit-ecommerce-api, specs/001-product-catalog/spec.md (commit e2cc35a)

예제

Step 3 · plan과 검증 게이트

/speckit.plan이 plan 템플릿을 복사하고, plan.md에는 Constitution Check 게이트가 있습니다

setup-plan.sh --json

```
$ .specify/scripts/bash/setup-plan.sh --json
{ "FEATURE_SPEC":".../spec.md",
  "IMPL_PLAN":".../plan.md",
  "SPECS_DIR":".../001-product-catalog" }
```

setup-plan.sh · commit 17bef8d

스크립트가 경로를 넘겨주면 LLM이 `plan.md`를 채우며 Phase 0·1 산출물(research·data-model·contracts·quickstart)도 함께 만듭니다. 위반이 있으면 **Complexity Tracking 표**에 근거를 적어야 진행할 수 있습니다.

plan.md — Constitution Check (실물) markdown

```
## Constitution Check
*GATE: Phase 0 전 통과, Phase 1 후 재확인.*

- I. Spec-Driven: spec.md 선행 ✓
- II. Test-First: 각 FR → vitest 시나리오 ✓
- III. Thin Storage: ProductStore
       인터페이스 뒤 인메모리 ✓
- IV. Explicit Errors: 400/404 + { error } ✓
- V. Server Authority: id를 서버가 생성 ✓

→ 위반 없음. Gate 통과.

## Complexity Tracking
위반 없음 — 표 비움.
```

plan.md · commit 17bef8d

살아있는 교재: speckit-ecommerce-api, plan.md Constitution Check (commit 17bef8d)

설명

Phase 1 설계 산출물

/speckit.plan은 plan.md 하나만 만들지 않고 설계를 관심사별로 나눕니다. 각 산출물을 독립적으로 리뷰·갱신할 수 있어, OpenSpec이 design.md 하나로 설계를 담는 것과 대비됩니다

1. **research.md — 기술 결정 (Phase 0)**
   웹 프레임워크=Hono(app.request()로 서버 없이 테스트), 검증=수동 순수 함수(zod는 1막에 과함), 저장=인메모리 Map+인터페이스, price=정수 cents(부동소수점 회피), id=crypto.randomUUID(). 명세의 NEEDS CLARIFICATION을 해소합니다.
   Phase 0

2. **data-model.md — 엔티티 (Phase 1)**
   Product(id·name·price·stock)의 필드·규칙·관계를 표로 정리하고, 검증 모드(생성=전체 필수, 수정=부분)까지 명시합니다.
   Phase 1

3. **contracts/ — API 계약 (Phase 1)**
   products.http.md에 5개 엔드포인트의 메서드·경로·요청 본문·성공/오류 코드를 표로 못박습니다. 구현과 테스트가 이 표를 직접 근거로 삼습니다.
   Phase 1

4. **quickstart.md — 종단간 검증 (Phase 1)**
   pnpm dev 후 curl로 201/200/404/400을 확인하는 절차. Success Criteria(SC-001~004)를 실제로 검증하는 스모크 테스트입니다.
   Phase 1

살아있는 교재: speckit-ecommerce-api, specs/001-product-catalog/ 실측 (commit 17bef8d)

실습

Step 4~5 · tasks와 implement

setup-tasks.sh가 설계 문서를 확인해 경로를 넘기면 LLM이 tasks.md를 쓰고, 작업은 Phase로 묶입니다

setup-tasks.sh --json

```
$ .specify/scripts/bash/setup-tasks.sh --json
{ "FEATURE_DIR":".../001-product-catalog",
  "AVAILABLE_DOCS":["research.md",
    "data-model.md","contracts/",
    "quickstart.md"],
  "TASKS_TEMPLATE":"..." }
```

setup-tasks.sh · commit ea17633

/speckit.implement 결과 확인

```
$ pnpm test
Test Files  4 passed (4)
     Tests  21 passed (21)
$ pnpm dev   # :3000 — quickstart curl
201 · 200 · 404 · 400 확인
```

commit 93daa21 · 869ffa9

tasks.md — Phase 분해 (실물 발췌) markdown

```
## Phase 2: Foundational
- [x] T003 Define Product, ProductStore,
  InMemoryProductStore in src/store.ts
- [x] T004 [P] validateProductInput(...)

## Phase 3: User Story 1 (P1)
- [x] T005 [P] [US1] tests POST /products
- [x] T006 [US1] POST /products
- [x] T008 [US1] GET /products, /products/:id
- Checkpoint: US1 독립 테스트 가능 — MVP

## Phase 4~5: US2 PATCH · US3 DELETE
## Phase 6: Polish — 21 tests + tsc + E2E
```

tasks.md · commit ea17633

실습

한 사이클 재현

git 태그로 완성본을, 커밋 해시로 각 단계를 되살려 초기화부터 구현까지 따라갑니다

재현

```
$ cd ~/frentis-repos/speckit-ecommerce-api
$ git checkout speckit-product-catalog
$ pnpm install
$ pnpm test
21 tests passed
$ pnpm dev   # :3000
quickstart.md의 curl로 종단간 확인
```

tag speckit-product-catalog

중간 단계는 `git checkout <커밋>`으로 되살립니다. 예: `git checkout e2cc35a`는 spec 작성 직후, `17bef8d`는 plan 완료 시점입니다.

단계별 커밋 지도 (실측)

단계	커밋	무엇을 남겼나
init	4cbb344	specify init — .specify/ · .claude/skills/
constitution	0b215bf	헌법 v1.0.0 (5원칙)
specify	e2cc35a	spec.md — User Story·FR·SC
plan	17bef8d	plan·research·data-model·contracts·quickstart
tasks	ea17633	tasks.md — Phase 분해
implement	93daa21·869ffa9	src/ 구현 + 21 tests

살아있는 교재: speckit-ecommerce-api, 태그 speckit-product-catalog · git log 실측

레퍼런스

Spec Kit 구현체·공식 문서 · CLI 버전과 설치 · 살아있는 교재 저장소 · 선행/후속 LO의 정확한 출처입니다

구현체 · GitHub

Spec Kit

SDD를 코딩 에이전트 위에서 구조화하는 도구입니다. CLI 이름은 specify이며, 워크플로우는 constitution → specify → plan → tasks → implement, 각 단계가 Markdown artifact를 산출합니다. 30여 개 AI 에이전트를 지원합니다.

예시: github.com/github/spec-kit · github.github.com/spec-kit

환경

버전 · 설치

Python 기반이라 uv tool로 설치합니다. 이 교재는 0.12.8 기준이며 실측 버전은 0.12.8.dev0입니다.

예시: uv tool install specify-cli --from git+https://github.com/github/spec-kit.git

저장소

살아있는 교재

관통 예제(상품 카탈로그 CRUD)의 실제 spec·plan·tasks·구현과 커밋·git 태그를 담은 저장소입니다. 콘솔·산출물은 모두 여기서 인용했습니다.

예시: ~/frentis-repos/speckit-ecommerce-api · 태그 speckit-product-catalog

학습 경로

선행 · 후속 LO

OpenSpec의 델타 변경 관리를 먼저 익히고, 다음으로 Spec Kit의 다중 feature 누적 구조로 나아갑니다.

예시: 선행: LO-OpenSpec 심화 delta 변경 관리 · 후속: LO-Spec Kit 심화 다중 feature

LO7 · SPEC KIT 기초 워크플로우

체크포인트

스스로 답할 수 있으면 LO7을 통과한 것입니다. 설치·초기화부터 CLI와 LLM의 분업, spec 구성, Constitution Check까지 여섯 가지를 점검합니다

01 설치 · 초기화 명령

uv tool install specify-cli --from git+... / specify init . --integration claude. specify --version은 0.12.8(실측 0.12.8.dev0).

02 다섯 단계 순서

constitution → specify → plan → tasks → implement. 사이 품질 게이트는 clarify·checklist·analyze.

03 CLI vs LLM 분업

create-new-feature.sh는 번호 폴더·spec 템플릿을 스캐폴드하고, 실제 spec 내용은 speckit-* 스킬(LLM)이 채웁니다.

04 User Story 우선순위

P1/P2/P3는 각각 독립 테스트 가능한 MVP 슬라이스. P1만으로도 최소 제품이 나와야 합니다.

05 Phase 1은 여러 파일

plan.md 하나가 아니라 research·data-model·contracts·quickstart로 관심사를 분리해 독립 리뷰합니다.

06 Constitution Check 게이트

Phase 0 전·Phase 1 후에 헌법 원칙을 점검. 위반 시 Complexity Tracking 표에 근거를 적어야 진행합니다.

✅ 기능마다 완결된 명세를 upfront로 갖추고, 헌법을 거버넌스 게이트로 둡니다. 이것이 Spec Kit 워크플로우의 핵심입니다.

SECTION

08 Spec Kit 심화 — 다중 feature

- 여러 feature(001·002·003)를 번호 붙은 폴더로 누적 관리하고, create-new-feature.sh가 번호를 자동으로 매기는 방식을 이해합니다
- 기존 동작을 바꿀 때 델타 없이 새 feature의 전체 명세로 표현하는 Spec Kit 방식을 OpenSpec의 MODIFIED 델타와 정면 비교합니다
- 003 재고 차감·취소의 spec·plan·data-model·tasks를 발췌해 방법론 주석과 [P]·[US] 포맷을 읽습니다
- Constitution 게이트의 반복과 checklist·analyze 두 품질 게이트를 구분하고, git 태그로 한 사이클을 재현합니다

50분 (블록 8)

개념

feature 폴더의 누적

기능마다 번호 폴더를 만들어, 지우지 않고 쌓아 올립니다

specs/ — 세 feature 폴더 (실물) text

```
specs/
├── 001-product-catalog/          # 상품 CRUD
├── 002-order-management/         # 주문(상품 참조·서버 총액)
└── 003-order-stock-cancellation/ # 재고 차감·취소(변경 관리)

각 폴더:
  spec · plan · research · data-model
  contracts/ · quickstart · tasks
```

ls specs/ · 각 폴더가 그 기능의 완결 명세

create-new-feature.sh — 번호 자동 증가

```
# 새 기능을 열면 다음 번호를 자동으로 매긴다
$ /speckit.specify Order stock and cancellation ...
- Scanning specs/ for highest number ... 002
Created specs/003-order-stock-cancellation/
Branch: 003-order-stock-cancellation
Next: /speckit.plan
```

create-new-feature.sh --json (L07 실측)

핵심 — 폴더는 **지우지 않고 누적**됩니다. 001을 만든 뒤 002를, 그다음 003을 더해도 앞 폴더는 그대로 남습니다. 각 폴더는 그 기능의 완결된 명세·설계·작업 문서를 담습니다.

실측 트리: speckit-ecommerce-api/specs/ (001·002·003), 태그 speckit-product-catalog · speckit-order-management · speckit-order-stock-cancellation

설명

세 feature 구조

이 교재의 살아있는 저장소는 세 feature로 진화합니다. 001이 상품을, 002가 주문(총액만)을, 003이 재고·취소를 더합니다. 003은 새 기능 추가이면서 동시에 002의 POST /orders 동작을 바꾸는 변경입니다

001	**product-catalog** 상품 CRUD — POST·GET·PATCH·DELETE /products	speckit-product-catalog 그린필드
002	**order-management** 주문 — 상품 참조·서버 총액 계산, 목록·단건 조회 (재고 미차감)	speckit-order-management 28 tests · 82bf4ca
003	**order-stock-cancellation** 재고 차감(부족 409)·취소 복원 — 002의 POST /orders 동작 변경	speckit-order-stock-cancellation 34 tests · 3d70db4

세 폴더가 **버전처럼 누적**됩니다. "지금의 진실"을 알려면 003의 spec까지 종합해야 합니다. 002의 spec은 재고 반영 이전(pre-stock) 버전의 역사적 기록으로 남습니다.

실측: speckit-ecommerce-api, git tag 3종 · 커밋 82bf4ca(002)·3d70db4(003)

개념

델타 없는 변경 방식

기존 동작을 바꾸는 변경도, 델타 없이 새 feature 전체 명세로 표현합니다

003이 하는 두 가지

① **동작 변경** — 주문 시 재고를 차감하고, 부족하면 409로 거절합니다(002의 POST /orders 개정).

② **기능 추가** — POST /orders/:id/cancel을 더해, 취소 시 차감했던 재고를 복원합니다.

OpenSpec이라면 order-management의 **Create Order 요구사항을 MODIFIED 델타**로 재작성하고, archive 시 하나의 main spec을 갱신했을 것입니다. Spec Kit은 다릅니다.

ℹ️ INFO **새 feature 003을 만든다**

그 spec에 재고·취소의 전체 명세를 씁니다. 002를 수정하지 않습니다.

create-new-feature.sh가 003 번호를 자동으로 매기고 폴더를 만듭니다.

💡 TIP **002의 spec은 그대로 둔다**

002는 재고 반영 이전(pre-stock) 버전의 역사적 기록으로 남습니다.

명세는 새 파일로 누적됩니다. 과거 spec은 지우지 않습니다.

⚠️ WARNING **코드는 진화, 명세는 누적**

구현은 store.ts·orders.ts를 수정하며 진화하지만, 명세는 새 feature 파일로 쌓입니다.

코드의 현재 상태와 spec 폴더의 누적이 분리됩니다.

LO 원문, Frentis 2026 · 근거: specs/003-order-stock-cancellation/spec.md (Relationship 절)

심화

OpenSpec 델타와 대조

같은 변경(재고·취소)을 두 도구가 어떻게 다르게 표현하는지가 핵심입니다. OpenSpec은 하나의 living spec을 델타로 진화시키고, Spec Kit은 feature 폴더를 스냅샷처럼 누적합니다. 어느 쪽이 옳다기보다 변경 성격에 따라 적합한 도구가 다릅니다

	OpenSpec — 연속성 레이어	Spec Kit — feature 스냅샷 누적
변경 표현	Create Order를 MODIFIED 델타로 재작성 → archive 시 main spec 하나를 갱신	새 feature 003의 전체 spec 작성, 002 spec은 불변
스펙 이력	openspec/specs/는 항상 최신 상태 하나(living), 변경 이력은 changes/archive/에	specs/001·002·003/ 폴더가 버전처럼 누적, 각자 전체 spec
"지금의 진실"	병합된 main spec 하나를 보면 됨	여러 feature spec을 종합해야 현재 동작을 앎
강점 영역	브라운필드·증분 변경 (연속성)	그린필드·기능별 완결 문서 (거버넌스)

예제

spec 개정 추적

델타가 없어, 무엇을 바꾸는지 spec 안 주석(Amends feature 002)으로 추적합니다

spec.md — Functional Requirements + Relationship (실물) markdown

```
### Functional Requirements
- FR-001: On order creation, System MUST verify
  sufficient stock for all items first, then
  decrement each product's stock. If any item is
  insufficient, reject with 409 and change nothing.
  *(Amends feature 002 Create Order.)*
- FR-003: System MUST cancel a pending order via
  POST /orders/:id/cancel: set status "cancelled"
  and restore the consumed stock (200).

## Relationship to feature 002 *(methodology note)*
This feature changes the behavior of POST /orders
from 002 (adds stock deduction) and adds
POST /orders/:id/cancel. In Spec Kit there is no
delta file — this feature's spec is a full
specification of the new behavior, while 002's spec
remains as the historical record of the pre-stock
version.
```

spec.md 발췌 · commit ba1acde

(Amends feature 002 Create Order.)

이 한 줄 주석이 개정 관계를 문서 안에 남깁니다. 델타 파일이 없으므로, 어느 기존 동작을 바꾸는지를 **주석과 별도 절**로 추적합니다.

Relationship 절 = 방법론 명문화

"In Spec Kit there is no delta file"라고 spec이 스스로 밝힙니다. 003의 spec은 바뀐 동작의 **전체 명세**이고, 002의 spec은 손대지 않으므로 pre-stock 버전의 역사가 보존됩니다.

실물: specs/003-order-stock-cancellation/spec.md, commit ba1acde(spec)

예제

data-model 확장 표기

표가 delta처럼 보이지만, spec 델타가 아니라 설계 문서 안의 변경 설명입니다

data-model.md (003) — 확장·재고 규칙 (실물) markdown

```
## Order (extends feature 002)
| field  | change                                |
|--------|---------------------------------------|
| status | 'pending' -> 'pending'|'cancelled'  |

- OrderStore gains setStatus(id, status).

## Product (reused from feature 001)
- stock decreases on order creation and increases
  on cancellation. No new field.

## 재고 조정 규칙
- 주문: aggregate(items)로 productId별 합산 ->
  전량 확인 -> 부족하면 409(변경 없음) -> 전량 차감.
- 취소: 각 수량을 상품 stock에 되돌린다.
  이미 cancelled면 409.
```

data-model.md 발췌 · Phase 1

"delta처럼 보이는 표" ≠ spec 델타

status 변화를 화살표로 적은 표는 **설계 문서 안의 변경 설명**입니다. OpenSpec의 MODIFIED 델타와 달리, 명세를 병합하는 장치가 아닙니다. 진실은 여전히 003의 전체 spec입니다.

재고 조정 규칙 = 원자성 설계

주문은 "합산 → 전량 확인 → 부족하면 409(변경 없음) → 전량 차감"으로, 취소는 역연산으로 정합성을 지킵니다. 이 규칙이 다음 슬라이드의 orders.ts 구현으로 그대로 이어집니다.

실물: specs/003-order-stock-cancellation/data-model.md

예제

재고 로직의 원자성

전량 확인 후 차감하는 코드는, FR-001의 문구를 그대로 옮긴 것입니다

src/orders.ts — POST /orders 재고 확인·차감 (실물) typescript

```typescript
// 3) 재고 확인(전량) — 하나라도 부족하면
//    아무것도 바꾸지 않는다
const required = aggregate(parsed)
for (const [productId, qty] of required) {
  const product = products.get(productId)!
  if (product.stock < qty) {
    return c.json(
      { error: `insufficient stock for ${productId}` },
      409,
    )
  }
}
// 4) 차감 후 주문 저장
for (const [productId, qty] of required) {
  const product = products.get(productId)!
  products.update(productId, { stock: product.stock - qty })
}
```

orders.ts 발췌 · commit 3d70db4

두 루프 분리 = 원자성

첫 루프가 **전량을 먼저 확인**하고, 통과해야만 둘째 루프가 **전량을 차감**합니다. 중간에 하나라도 부족하면 409로 빠져나가 어떤 재고도 손대지 않습니다.

원칙 IV·V가 코드에 드러난다

원칙 IV(Explicit Errors) — 재고 부족은 **409**로 명시합니다. 원칙 V(Server Authority) — 서버가 재고를 조정합니다. 중복 item은 `aggregate()`로 productId별 합산해 확인·차감합니다.

실물: src/orders.ts, commit 3d70db4(003 구현) · 태그 speckit-order-stock-cancellation

설명

tasks 포맷 — Phase·[P]·[US]

tasks.md는 작업을 세 축으로 조직합니다. 작업 ID는 T001부터 순번을 매기고 설명에 정확한 파일 경로를 넣습니다. Phase 사이에는 Checkpoint를 두어 여기까지 하면 이 스토리를 독립적으로 검증할 수 있다를 표시합니다

Phase	**실행 단계** Setup → Foundational → User Story 1 → 2 → … → Polish. Foundational은 모든 User Story를 막는 선행 작업(blocking)입니다.
[P]	**병렬 가능** 서로 다른 파일을 만지고 의존이 없어 동시에 진행할 수 있는 작업(주로 테스트 작성). 같은 파일을 만지는 구현 작업은 순차로 둡니다.
[US1] / [US2]	**User Story 소속** 그 작업이 어느 User Story 소속인지. 작업을 스토리로 묶어 각 스토리를 독립적으로 구현·테스트·데모할 수 있게 합니다(추적성).

003처럼 기존 feature를 확장하는 경우, tasks는 **Setup 없이 Foundational**(store.ts의 status union·setStatus 추가)부터 시작해 US1(재고 차감)·US2(취소)로 이어집니다.

예제

병렬과 순차의 구분

[P]가 붙은 테스트 작업은 병렬 가능하고, 같은 파일을 만지는 작업은 순차입니다

tasks.md (003) — Phase·[P]·[US] (실물) markdown

```
## Phase 2: Foundational
- [ ] T001 Change Order.status to
      'pending'|'cancelled' and add
      OrderStore.setStatus in src/store.ts

## Phase 3: User Story 1 — Reserve stock (P1)
- [ ] T002 [P] [US1] Write tests: create
      decrements stock; insufficient -> 409
- [ ] T003 [US1] Update POST /orders to
      check-all-then-deduct stock (409)
- Checkpoint: US1 독립 테스트 가능 — 재고 반영

## Phase 4: User Story 2 — Cancel & restore (P2)
- [ ] T004 [P] [US2] Write tests: cancel restores
      stock (200); missing 404; re-cancel 409
- [ ] T005 [US2] Implement POST /orders/:id/cancel
```

tasks.md 발췌 · T001–T005

T002·T004 = [P] 병렬

테스트 파일을 만지므로 서로 의존이 없어 **동시에** 작성할 수 있습니다.

T003·T005 = [P] 없음 순차

같은 `orders.ts`를 만지는 구현이라 앞 테스트에 의존해 순차입니다. Foundational(T001)이 US1·US2를 모두 막는 선행 작업입니다.

세 라벨(Phase·[P]·[US])이 다중 feature에서 **작업 병렬성과 스토리 독립성**을 동시에 관리합니다.

실물: specs/003-order-stock-cancellation/tasks.md · Foundational→US1→US2→Polish

심화

Constitution 게이트의 반복

Constitution은 프로젝트에 하나지만, 게이트는 feature마다 반복됩니다. 003의 plan.md도 Constitution Check를 거칩니다. 헌법이 모든 feature의 계획 앞에 놓인 공통 관문이라는 점이 다중 feature에서 드러납니다

plan.md (003) — Constitution Check (실물) markdown

```
## Constitution Check
*GATE.* I~V ✓ — 특히
  IV Explicit Errors (409 conflict),
  V Server Authority (서버가 재고 조정).
위반 없음.

## Complexity Tracking
위반 없음 — 비움.
```

plan.md 발췌 · 위반 없어 통과

헌법 하나, 게이트는 feature마다

001·002·003의 plan.md가 각각 Constitution Check를 통과해야 합니다. 001에서 통과했다고 003이 면제되지 않습니다. 계획 단계마다 같은 헌법을 다시 대조합니다.

003에서 직접 관련되는 원칙

원칙 IV(Explicit Errors) — 재고 부족·중복 취소에 **409**. 원칙 V(Server Authority) — 서버가 재고를 조정. 위반이 없으므로 Complexity Tracking을 비우고 통과합니다.

실물: specs/003-order-stock-cancellation/plan.md (Constitution Check · Complexity Tracking)

심화

품질 게이트 두 층

checklist는 한 문서의 품질을, analyze는 여러 문서 사이의 정합을 점검합니다

`/speckit.checklist`

요구사항 글의 품질

"요구사항 글쓰기의 단위 테스트(Unit Tests for English)". 코드나 구현이 아니라 **요구사항 자체의 품질**(명료성·완전성·모호성 없음)을 점검합니다.

"버튼이 잘 눌리는지"가 아니라 **"이 요구사항이 명확하게 쓰였는지"**를 봅니다. plan 이후 spec의 완성도를 높이는 데 씁니다.

`/speckit.analyze`

산출물 사이의 일관성

task 생성 후 실행하는 **비파괴 교차 산출물 일관성 분석**.
spec.md·plan.md·tasks.md가 서로 어긋나지 않는지 점검합니다.

예: spec의 FR-003(취소=200·상태·복원)이 tasks의 T005와 quickstart의 취소 검증에 **모두 반영됐는지** 확인합니다.

checklist = 한 문서(요구사항)의 품질

analyze = 여러 문서 사이의 일관성

둘 다 선택이지만, feature가 쌓일수록 analyze의 가치가 커집니다.

실습

두 feature 재현

살아있는 교재입니다. git checkout speckit-order-management로 002(28 tests)를, speckit-order-stock-cancellation으로 003(34 tests)을 그대로 되살립니다. quickstart로 재고 차감·취소 복원·부족 409를 종단간 확인합니다

bash — 두 feature 완료 시점 재현

```
# 002 완료 — 주문(총액만), 재고 미차감
$ git checkout speckit-order-management
HEAD → 82bf4ca  (28 tests, pre-stock)

# 003 완료 — 재고 차감·취소 복원
$ git checkout speckit-order-stock-cancellation
HEAD → 3d70db4  (34 tests)
$ pnpm install && pnpm test
Test Files  4 passed  ·  Tests  34 passed
```

git tag 2종 · pnpm test 34 passed

quickstart(003) — 재고 차감·복원·409 E2E

```
# 재고 5로 상품 등록 → 2개 주문
$ curl localhost:3000/products/$PID
stock: 3          # 2개 차감됨
# 주문 취소 → 재고 복원
$ curl -X POST .../orders/$OID/cancel
200  status: cancelled
$ curl localhost:3000/products/$PID
stock: 5          # 복원됨
# 재고 초과 주문·재취소
409 insufficient stock  /  409 already cancelled
```

quickstart.md · SC-001~SC-004

실측: speckit-ecommerce-api, 태그 speckit-order-management·speckit-order-stock-cancellation · quickstart.md(003)

LO8 · SPEC KIT 심화: 다중 FEATURE

레퍼런스

Spec Kit 1차 출처(GitHub) · 살아있는 교재 저장소와 태그 · OpenSpec 대조 짝 · 선행 LO의 정확한 출처입니다

구현체 · GitHub

Spec Kit

GitHub의 SDD 도구입니다. 워크플로우는 Constitution → Specify → Plan → Tasks → Implement이며, 각 phase가 Markdown artifact를 산출합니다. 30여 개 AI 에이전트를 지원합니다.

예시: github.com/github/spec-kit · github.github.com/spec-kit · specify CLI

실습 저장소

살아있는 교재

003 재고·취소까지 실제로 개발한 저장소입니다. specs/001·002·003의 spec·plan·data-model·tasks와 src/store.ts·orders.ts가 근거입니다.

예시: speckit-ecommerce-api · 태그 speckit-order-management(002)·speckit-order-stock-cancellation(003)

선택 게이트

품질 게이트

checklist는 요구사항 글의 품질(영어의 단위 테스트)을, analyze는 spec·plan·tasks 교차 일관성을 점검합니다. 둘 다 선택이며 feature가 쌓일수록 analyze가 중요해집니다.

예시: /speckit.checklist · /speckit.analyze

관련 학습

대조 짝 · 선행 LO

OpenSpec은 MODIFIED 델타로 하나의 living spec을 진화시킵니다(짝 트랙). 선행 학습은 단일 feature의 Spec Kit 워크플로우입니다.

예시: openspec-vs-speckit.md · LO-OpenSpec 심화 delta 변경 관리 · LO-Spec Kit 기초 워크플로우

LO8 · SPEC KIT 심화: 다중 FEATURE

체크포인트

스스로 답할 수 있으면 LO8을 통과한 것입니다. 델타 없는 변경 표현부터 두 품질 게이트까지 여섯 가지를 점검합니다

01 기존 동작 변경 표현

새 feature 폴더 + 전체 명세로 표현합니다. OpenSpec은 MODIFIED 델타 + main spec 병합입니다.

02 002 spec의 운명

그대로 둡니다(불변). pre-stock 버전의 역사적 기록으로 보존하기 위함입니다.

03 "지금의 진실"

OpenSpec은 main spec 하나로, Spec Kit은 여러 feature spec을 종합해야 현재 동작을 압니다.

04 [P]와 [US1]

[P]=다른 파일·병렬 가능(주로 테스트), [US1]=User Story 1 소속·추적성을 뜻합니다.

05 checklist vs analyze

checklist=요구사항 글 품질, analyze=spec·plan·tasks 교차 일관성을 봅니다.

06 도구 선택 기준

브라운필드 증분 변경이 잦으면 OpenSpec의 델타(연속성), 기능별 완결 문서·거버넌스면 Spec Kit(스냅샷 누적).

✅ Spec Kit에는 델타가 없습니다. 변경도 새 feature의 전체 명세로 누적하고, 과거 spec은 역사로 남깁니다.

SECTION

09 두 구현체 정면 비교 — OpenSpec 대 Spec Kit

- 같은 요구사항을 두 도구로 구현한 실물 산출물을 나란히 놓고 차이의 근원을 짚습니다
- 초기화·워크플로우·변경 관리·스펙 조직 네 축에서 무엇이 어떻게 다른지 대조표로 확인합니다
- 변경 관리(MODIFIED 델타 대 새 feature 폴더)를 두 repo의 실제 파일로 정면 대조합니다
- 브라운필드·그린필드, 변경 빈도, 감사 요구, 팀 성숙도에 따라 선택 프레임으로 판단합니다

50분 (블록 9)

개념

비교의 목적

같은 API를 두 도구로 만들어, 산출물이 어떻게 달라지는지 봅니다

같은 요구사항,
다른 산출물

비교의 근거는 추측이 아니라 같은 머신에서 실제로 초기화하고 끝까지 구현한 산출물입니다. 두 도구는 같은 SDD 방법론의 서로 다른 구현체입니다.

1 **같은 도메인, 두 트랙으로 구현**
상품 CRUD와 주문·재고·취소를 OpenSpec과 Spec Kit으로 각각 만들었습니다. 요구사항은 한 벌입니다.

2 **차이는 취향이 아니라 정체성**
OpenSpec은 하나의 living spec을 델타로 진화시키는 연속성 레이어, Spec Kit은 기능별 완결 명세를 쌓는 거버넌스 레이어입니다.

3 **네 축으로 대조**
초기화, 워크플로우, 변경 관리, 스펙 조직. 이 네 축에서 두 도구가 어떻게 갈리는지 실물로 확인합니다.

4 **결론은 선택 프레임**
"어느 게 더 좋은가"가 아니라 "우리 프로젝트가 무엇을 자주 하는가"가 기준입니다.

설명

초기화 산출물 비교

처음 붙일 때 무엇이 생기는지가 정체성을 드러냅니다 — 실행 중 계산이냐, 파일로 까느냐

기능	OpenSpec	Spec Kit
설정·헌법 한 파일 config.yaml 대 constitution.md	✓	✓
템플릿을 파일로 배치 .specify/templates/* (spec·plan·tasks)	—	✓
스캐폴드 bash 스크립트 create-new-feature·setup-plan	—	✓
CLI가 상태·지시를 계산 status·instructions (artifact-guided)	✓	—
워크플로우 정의 파일 config schema 대 workflow.yml	—	✓
슬래시·스킬 세트 .claude/commands/opsx 대 skills/speckit-*	✓	✓

핵심 대비는 **'CLI가 계산하느냐, 파일로 존재하느냐'**입니다. OpenSpec은 가볍게 시작하고 나머지는 CLI가 실행 중에 계산합니다. Spec Kit은 처음부터 구조를 파일로 세워둡니다.

실측: openspec init --tools claude · specify init --integration claude · openspec-vs-speckit.md

예제

제약이냐 문서냐

config.yaml은 AI에게 주입되는 제약이고, constitution.md는 사람이 읽는 문서입니다

OpenSpec · config.yaml (두뇌용 제약)

openspec/config.yaml — 발췌 markdown

```
schema: spec-driven

context:
  stack: TypeScript, Hono, pnpm, vitest
  domain: e-commerce product/order API
  conventions: Conventional Commits

rules:
  proposal: What/Why만, How는 design에서
  specs: Requirement + #### Scenario (4-hashtag)
  design: 데이터 모델·인터페이스 결정
```

config.yaml · 산출물에는 미포함

Spec Kit · constitution.md (거버넌스 문서)

.specify/memory/constitution.md — 발췌 markdown

```
# E-Commerce API Constitution

## Core Principles
### I. Spec-Driven (NON-NEGOTIABLE)
Every feature begins as a spec (WHAT / WHY)
before any code.

### II. Test-First
Each functional requirement maps to at least
one test. Failure paths (400/404/409) are
specified and tested, not only happy paths.

### V. Server Authority
The server computes derived values (totals,
stock). Client-supplied fields are ignored.
```

constitution.md · 리뷰 대상 문서로 잔존

실물: openspec/config.yaml · .specify/memory/constitution.md (speckit-ecommerce-api)

설명

워크플로우 단계 비교

OpenSpec의 3단계는 순환이 빠르고, Spec Kit의 5~6단계는 단계마다 산출물을 남깁니다

OPENSPEC · 3단계

propose → apply → archive

- 산출물: proposal / specs(delta) / design / tasks
- 설계 문서는 design.md 한 파일에 담습니다
- CLI(new·status·instructions·validate·archive)가 상태와 지시를 계산
- 품질 게이트: validate --strict, verify(3C)
- 짧고 순환이 빨라 증분 반복에 유리합니다

대

SPEC KIT · 5~6단계

constitution → specify → plan → tasks → implement

- 산출물: spec / plan / tasks (+ data-model·contracts·research·quickstart)
- 설계를 데이터 모델·API 계약·리서치·빠른 시작으로 분할
- init + bash 스캐폴드, 나머지는 슬래시 커맨드(LLM 주도)
- 선택 게이트: clarify·checklist·analyze
- 문서가 많아 팀 리뷰·감사에 유리, 빠른 반복엔 부담

실측: openspec-vs-speckit.md · 두 repo의 실제 단계·산출물 구조

심화

변경 관리 방식

같은 변경을 OpenSpec은 MODIFIED 델타로, Spec Kit은 새 feature 폴더로 처리합니다

관점	OpenSpec	Spec Kit
변경 표현	Create Order를 MODIFIED 델타로 재작성, 취소는 ADDED	새 feature 003의 전체 spec 작성, 기존 002는 그대로
병합 결과	archive 시 main spec 파일 하나가 갱신됨	specs/003 폴더가 추가, 002는 재고 이전 상태로 잔존
스펙 이력	specs/는 항상 최신 하나(living), 이력은 changes/archive/	001·002·003… 폴더가 버전처럼 누적, 각자 전체 spec
지금의 진실	병합된 main spec 하나만 보면 됨	여러 feature spec을 종합해야 현재 동작을 알 수 있음

한 문장 요약 — OpenSpec은 하나의 스펙을 델타로 진화시키고(연속성), Spec Kit은 기능별 스냅샷을 누적합니다(거버넌스).

실물: ops-modified-delta.txt(MODIFIED/ADDED) · specs/003-order-stock-cancellation/

예제

같은 변경의 실물 대조

코드 진화는 둘 다 비슷하지만, 스펙을 남기는 방식이 다릅니다

OpenSpec · MODIFIED 델타 → main spec 하나 갱신

specs/order-management/spec.md (델타 발췌) — markdown

```markdown
## MODIFIED Requirements

### Requirement: Create Order
The system SHALL ... decrement each
product's `stock` by the ordered quantity ...

#### Scenario: Reject insufficient stock
- **WHEN** available `stock` < requested
- **THEN** responds `409`
- **AND** no product's stock is changed

## ADDED Requirements

### Requirement: Cancel Order
... set `status` to `"cancelled"`, and
restore the stock the order consumed.
```

ops-modified-delta.txt · 한 파일에 병합

Spec Kit · 새 feature 003 폴더 (002는 그대로)

ls specs/

```
$ ls specs/
001-product-catalog/
002-order-management/      # 재고 이전 그대로
003-order-stock-cancellation/  # 새로 추가

$ ls specs/003-order-stock-cancellation/
spec.md  plan.md  tasks.md
data-model.md  quickstart.md

# 현재 동작을 알려면 002와 003을
# 함께 읽어야 합니다.
```

spk-features.txt · 폴더 누적

실물: openspec-ecommerce-api(act2 델타) · speckit-ecommerce-api(specs/003)

설명

스펙 조직 비교

OpenSpec은 capability 단위 하나씩, Spec Kit은 feature 폴더가 번호순으로 쌓입니다

항목	OpenSpec	Spec Kit
활성 작업 위치	openspec/changes/<name>/	specs/NNN-<name>/ (번호 폴더)
완료된 스펙	specs/<capability>/spec.md — capability별 단일 living spec	specs/NNN-*/spec.md — feature별 전체 spec이 그대로 잔존
이력 보관	changes/archive/YYYY-MM-DD-<name>/	폴더 누적 자체가 이력
요구사항 표현	Requirement + Scenario (WHEN/THEN, #### 4-hashtag)	User Story 우선순위(P1/P2/P3) + FR-001~ + Acceptance Scenario

OpenSpec — 'product-catalog는 5개 requirement'처럼 능력마다 최신 상태 파일 하나.

Spec Kit — 001, 002, 003... 폴더가 번호순으로 계속 쌓입니다.

실물: openspec/specs/<capability>/spec.md · specs/NNN-<name>/spec.md

예제

같은 기능, 두 표현

같은 상품 등록·조회 요구를 OpenSpec은 Requirement와 Scenario로, Spec Kit은 User Story로 표현합니다

OpenSpec · Requirement + Scenario

specs/product-catalog/spec.md (실물) — markdown

```
### Requirement: Create Product
The system SHALL create a product from
`name`, `price` (integer cents), and
`stock`, assign a unique `id`, and return
the created product.

#### Scenario: Create with valid input
- **WHEN** POST /products with
  { "name": "Keyboard", "price": 4200 }
- **THEN** responds `201`
- **AND** body contains a generated `id`

#### Scenario: Reject missing name
- **WHEN** POST /products without `name`
- **THEN** responds `400`
```

병합된 단일 spec · 5 requirement

Spec Kit · User Story 우선순위 + Acceptance

specs/001-product-catalog/spec.md (실물) — markdown

```
### User Story 1 – Register and browse
    products (Priority: P1)

**Why this priority**: 상품이 없으면 주문도
불가능하다. 등록·조회는 카탈로그의 최소
기능이며 독립적으로 가치를 제공한다.

**Independent Test**: 상품을 등록한 뒤
목록·단건 조회로 보이는지 확인.

**Acceptance Scenarios**:
1. **Given** 빈 카탈로그, **When** 등록,
   **Then** 식별자와 함께 반환 (201).
4. **Given** 없는 식별자, **When** 조회,
   **Then** 찾을 수 없음 (404).
```

feature 폴더 · P1/P2/P3 + FR-001~

실물: openspec/specs/product-catalog/spec.md · specs/001-product-catalog/spec.md

개념

연속성 대 거버넌스

어느 쪽이 우월한 게 아니라, 무엇을 자주 하느냐가 도구를 정합니다

OPENSPEC

연속성 레이어

- 하나의 living spec을 델타로 진화시킵니다
- 기존 코드를 조금씩 고치는 브라운필드·증분 변경에 강합니다
- "지금의 진실"이 한 곳에 모여 있어 상태 파악이 쉽습니다
- 3단계·설정 한 파일로 학습 곡선이 완만합니다

대

SPEC KIT

거버넌스 레이어

- 기능별 upfront 종합 명세(spec/plan/tasks)를 둡니다
- 새로 만드는 그린필드·구조적 진행에 강합니다
- 각 기능이 완결 문서로 남아 리뷰·감사·규제 대응에 유리합니다
- clarify·analyze 품질 게이트로 정합성을 점검합니다

심화

브라운필드 대 그린필드 적합성

상황이 도구를 정합니다. 어느 한쪽을 편들지 않고, 프로젝트가 무엇을 자주 하는지에 따라 유리한 도구와 그 근거를 정리합니다

상황	유리한 도구	이유
이미 돌아가는 서비스를 조금씩 고침	OpenSpec	델타로 바뀐 부분만 명세, main spec이 항상 현재를 반영
새 제품·새 모듈을 처음부터 만듦	Spec Kit	기능별 완결 명세와 단계별 품질 게이트가 그린필드 설계에 맞음
변경이 잦고 스펙 상태를 한눈에 봐야 함	OpenSpec	단일 living spec — 한 파일이 곧 지금의 진실
기능마다 독립 문서·리뷰·감사 근거가 필요	Spec Kit	번호 feature 폴더가 각각 완결 문서로 남음
팀이 아직 SDD에 익숙하지 않아 가볍게 시작	OpenSpec	3단계, 설정 한 파일로 학습 곡선이 완만
여러 명이 각자 기능을 맡아 병렬 개발	Spec Kit	feature 폴더로 격리, constitution으로 공통 규율

실습

선택 프레임

네 질문에 답하면 도구가 정해집니다. 두 트랙을 같은 시점으로 체크아웃해 스펙 파일을 직접 열고, 설계 문서 수와 요구사항 표현을 비교합니다

1. 주로 고치나, 주로 만드나?
기존 코드 증분 변경이 잦으면 OpenSpec, 새 기능을 완결해 쌓으면 Spec Kit.

2. "지금의 진실"이 한 곳에 모여야 하나?
예면 OpenSpec(단일 living spec), 흩어져도 되면 Spec Kit.

3. 리뷰·감사·규제 근거가 중요한가?
중요하면 Spec Kit(완결 문서 + 게이트), 속도가 우선이면 OpenSpec.

4. 팀 성숙도는?
입문·소규모면 OpenSpec, 구조·거버넌스가 필요한 규모면 Spec Kit.

산출물 나란히 열기

```
# OpenSpec — 상품 카탈로그 완료 시점
$ cd $FRENTIS_REPOS/openspec-ecommerce-api
$ git checkout act1-product-catalog
$ cat openspec/specs/product-catalog/spec.md
→ 단일 spec.md 하나

# Spec Kit — 같은 기능
$ cd $FRENTIS_REPOS/speckit-ecommerce-api
$ git checkout speckit-product-catalog
$ ls specs/001-product-catalog/
→ spec.md plan.md data-model.md contracts/ …
```

설계 문서 수를 세어 비교

결론이 '혼용'일 수도 있습니다 — 레거시는 OpenSpec으로 증분, 신규 제품은 Spec Kit으로 그린필드.

실습: git checkout act1-product-catalog / speckit-product-catalog (두 repo)

LO9 · 두 구현체 정면 비교

레퍼런스

두 구현체의 1차 출처(GitHub·공식 문서) · 버전 · 실측 비교 문서 · 선행 LO의 정확한 출처입니다

구현체 · Fission-AI

OpenSpec

하나의 living spec을 spec delta(ADDED/MODIFIED/REMOVED)로 진화시키는 연속성 레이어입니다. CLI가 상태·지시를 계산하는 artifact-guided 방식입니다.

예시: github.com/Fission-AI/OpenSpec · openspec.dev · npm @fission-ai/openspec v1.5.0

구현체 · GitHub

Spec Kit

기능별 완결 명세와 품질 게이트를 쌓는 거버넌스 레이어입니다. Constitution→Specify→Plan→Tasks→Implement 워크플로우로 각 phase가 Markdown artifact를 남깁니다.

예시: github.com/github/spec-kit · github.github.com/spec-kit/ · specify CLI 0.12.8 (2026-07 확인)

살아있는 교재

실측 비교 문서

두 도구를 같은 머신에서 초기화·구현한 산출물로 대조한 문서입니다. 초기화·워크플로우·변경 관리를 파일 구조 실측으로 비교합니다.

예시: speckit-ecommerce-api/docs/openspec-vs-speckit.md · 두 repo git 태그(act1/act2, speckit-*)

SDD 실습

선행 LO

두 도구를 각각 기초·심화로 익힌 LO들입니다. OpenSpec 심화(delta), Spec Kit 심화(다중 feature)를 거쳐 이 비교에 도달합니다.

예시: LO-OpenSpec 심화 · LO-Spec Kit 심화 다중 feature (Frentis 2026)

LO9 · 두 구현체 정면 비교

체크포인트

스스로 답할 수 있으면 LO9를 통과한 것입니다. 정체성부터 선택 프레임까지 다섯 가지를 점검합니다

01 두 도구의 정체성

OpenSpec은 연속성 레이어(living spec을 델타로 진화), Spec Kit은 거버넌스 레이어(기능별 완결 명세·품질 게이트). 한 문장씩 말할 수 있습니까?

02 같은 변경, 다른 처리

재고 차감 추가를 OpenSpec은 MODIFIED 델타로 main spec 하나를 갱신, Spec Kit은 새 feature 003 폴더로. 어떻게 다른지 설명할 수 있습니까?

03 지금의 진실은 어디에

OpenSpec은 병합된 main spec 한 파일, Spec Kit은 002·003을 함께 읽어 재구성. 각각 어디서 확인하는지 말할 수 있습니까?

04 스펙 조직 차이

capability별 단일 living spec 대 번호 feature 폴더 누적. 요구사항 표현(Requirement/Scenario 대 User Story P1/P2/P3)을 구분할 수 있습니까?

05 선택 프레임 적용

브라운/그린필드, 변경 빈도, 감사 요구, 팀 성숙도에 따라 네 질문으로 어느 도구가 유리한지 판단할 수 있습니까?

⚖ 선택 기준은 "어느 게 더 좋은가"가 아니라 "우리 프로젝트가 무엇을 자주 하는가"입니다. 혼용도 현실적인 선택입니다.

SECTION

10 조직 도입 로드맵과 거버넌스

- SDD 성숙도 단계(개인 → 팀 → 조직)를 구분하고 진단 질문으로 우리 조직의 현재 단계를 판단합니다
- 팀 컨벤션(CLAUDE.md/constitution)·AI 코드 리뷰·스펙 거버넌스(변경 이력·감사)를 조직 규율로 설계합니다
- Phase 1~3 도입 로드맵을 세우고 각 단계의 리스크와 대응책을 미리 마련합니다

50분 (블록 8)

개념

도구 숙달과 조직 정착의 차이

개인이 잘 쓰는 것과 팀이 일관되게 쓰는 것은 다릅니다

도구를 깐다고 SDD가 도입되지 않습니다

규율이 **문서로 남고, 리뷰로 강제되고, 이력으로 감사**될 때 비로소 조직에 정착합니다. 개인의 습관은 팀으로 저절로 번지지 않습니다.

속도가 빨라진 만큼 위험도 커집니다

AI가 코드를 빠르게 생산할수록 **잘못된 방향으로도 빠르게** 갑니다. SDD는 의도(스펙)를 진실의 원천으로 삼아 AI를 통제하는 방법론이고, 도입의 핵심은 이 통제 장치를 팀에 심는 것입니다.

ℹ️ INFO **간극의 실체**

개인 → 팀 → 조직으로 갈수록 컨벤션·리뷰·거버넌스라는 세 겹의 벽이 생깁니다. 이 LO는 그 벽을 어떻게 넘는지를 다룹니다.

"혼자서는 잘 되는데 남에게 전달이 안 된다"가 바로 첫 벽의 신호입니다.

💡 TIP **핵심 자산은 코드가 아니라 스펙**

스펙이 팀 컨벤션·리뷰·이력의 중심에 서야 AI가 코드를 빠르게 써내도 품질과 통제를 잃지 않습니다.

이후 슬라이드는 스펙을 규율의 중심에 놓는 네 가지 장치(컨벤션·리뷰·거버넌스·로드맵)를 순서대로 봅니다.

설명

도입 성숙도 3단계

도입은 전사에 한 번에 미는 게 아니라, 성숙도를 밟아 올라가는 여정입니다

단계	스펙의 위상	리뷰	거버넌스	넘어갈 신호
L1 개인	개인 작업 메모 수준	셀프 리뷰	없음	혼자서는 잘 되는데 남에게 전달이 안 됨
L2 팀	팀 공유 산출물, 컨벤션 문서화	동료 스펙 리뷰	변경 이력 관리	스펙 없이 짠 코드를 팀이 거부하기 시작함
L3 조직	조직 표준·감사 대상	AI와 사람 이중 리뷰	감사·규제 대응, 지표화	스펙 품질이 조직 성과 지표에 들어감

L1 개인
혼자 도구를 익히는 단계. 여기서 멈추면 "저 사람만 잘 쓰는 도구"로 끝납니다.

L2 팀
컨벤션이 문서로 서고, 동료가 스펙을 리뷰합니다. 스펙 없는 습관을 팀이 거부하면 안착입니다.

L3 조직
스펙이 조직 표준이 되고 변경 이력이 감사 대상이 됩니다. AI 리뷰가 프로세스에 들어옵니다.

심화

도입 진단 질문

스펙 없는 PR을 반려하면 최소 L2, 아무도 스펙을 안 쓰면 아직 L1입니다

아직 L1 개인

스펙을 아무도 안 쓴다

- ✕ 스펙 없이 올라온 PR을 아무도 반려하지 않습니다
- ✕ 스펙은 개인 메모에 가깝고 남이 이어받기 어렵습니다
- ✕ 여기서 멈추면 저 사람만 잘 쓰는 도구로 끝납니다
- ✕ 팀 컨벤션이 문서로 존재하지 않습니다

진단

최소 L2 팀

스펙 없는 코드를 팀이 거부한다

- ✓ 스펙 없이 올라온 PR을 리뷰어가 반려합니다
- ✓ 팀 컨벤션이 CLAUDE.md/constitution 문서로 섭니다
- ✓ 스펙을 동료가 리뷰하기 시작합니다
- ✓ 스펙 먼저 짜는 습관이 팀에 안착했습니다

설명

팀 컨벤션

우리 팀 규칙이 문서로 있어야 일관됩니다 — config.yaml과 constitution이 그 자리입니다

	OpenSpec	Spec Kit	공통 목적
규율 문서	openspec/config.yaml의 context·rules	.specify/memory/constitution.md	팀 공통 규율을 코드 밖에 고정
Claude Code 규율	CLAUDE.md	CLAUDE.md	AI 에이전트에게 팀 컨벤션 주입

CLAUDE.md(또는 constitution)에 담을 팀 컨벤션 다섯 가지

01
기술 스택·아키텍처 원칙
언어·프레임워크·계층 구조·금지 패턴

02
스펙 작성 규칙
모든 requirement에 실패 시나리오, 검증(400)을 존재 확인(404)보다 먼저

03
테스트 규율
TDD 순서·커밋 단위·테스트 없는 구현 금지

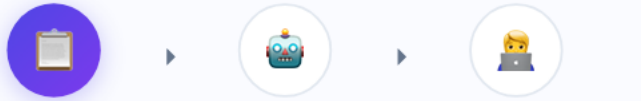
04
리뷰 게이트
스펙 리뷰 통과 전 구현 착수 금지

05
금지 사항(Non-goals)
AI가 임의로 확장하면 안 되는 범위

설명

삼중 리뷰 구조

사람이 모든 라인을 읽으면 리뷰가 병목이라, 스펙 리뷰를 먼저 두고 AI 리뷰로 걸러냅니다



1차 스펙 리뷰 (사람)

의도가 맞나 — requirement·시나리오가 실션 요구를 반영하는지 구현 전에 봅니다. 의도가 틀리면 코드가 깨끗해도 소용없습니다.

2차 AI 코드 리뷰 (AI)

스펙 준수·명백한 버그·컨벤 위반을 1차로 걸러냅니다. 복잡 판단은 Opus 4.8, 일반 리뷰는 Sonnet 4.6.

3차 사람 코드 리뷰 (사람)

AI가 놓친 판단·설계·보안, 리스크 높은 변경에 사람 리뷰를 몰아줍니다.

⚠️ AI 리뷰 결과를 무비판적으로 적용하지 않습니다. AI의 지적도 검증 대상이며, 근거 없는 지적은 반려합니다.

심화

스펙 거버넌스

SDD의 자산은 스펙입니다. 스펙이 어떻게 바뀌어 왔는지 추적할 수 있어야 조직 차원의 거버넌스가 성립합니다. 두 도구의 이력 구조가 감사에 각자 유리한 면이 있습니다

관점	OpenSpec	Spec Kit
현재 상태	단일 living spec (openspec/specs/)	feature 폴더 누적 (specs/NNN-*/)
변경 이력	changes/archive/YYYY-MM-DD-<name>/에 델타 보존	폴더 누적 자체가 시점별 스냅샷
감사 관점	지금의 진실을 한 파일로 확인 + 아카이브에서 변경 추적	각 feature가 완결 문서라 시점별 근거가 명확

거버넌스로 확정할 네 가지

변경 승인 절차
스펙 변경(델타·새 feature)이 누구의 리뷰·승인을 거치는지 정합니다.

이력 보존 규칙
아카이브(OpenSpec)나 feature 폴더(Spec Kit)를 지우지 않고 보존합니다.

감사 접근
규제·보안 감사 시 "이 기능이 왜 이렇게 동작하는가"를 스펙 이력으로 증빙합니다.

버전·릴리스 연동
스펙 변경과 코드 릴리스를 git 태그로 묶습니다(교재의 act1-*, speckit-* 태그).

설명

도입 로드맵 3단계

전사 전환을 한 번에 하면 실패합니다. 파일럿 → 표준화 → 확산으로 나눕니다



Phase 1 · 증명

자원자 팀으로 작게 시작해 실제로 된다를 증명합니다. 학습 곡선이 완만한 OpenSpec을 권합니다.

완료 신호: 스펙 없이 짠 코드를 팀이 거부

Phase 2 · 표준화

파일럿의 교훈으로 컨벤션을 다듬고 인접 팀으로 넓힙니다. AI 코드 리뷰를 이 단계에서 붙입니다.

완료 신호: 팀 간 스펙 리뷰가 자연스러움

Phase 3 · 정착

조직 표준·거버넌스·감사가 자리 잡습니다. 레거시는 OpenSpec, 신규는 Spec Kit 같은 혼용 정책을 확정합니다.

완료 신호: 스펙 품질이 조직 지표에 들어감

심화

도입 리스크와 대응

리스크의 공통 뿌리는 '도구를 규율 없이 도입하는 것'입니다. 대응의 공통 축은 문서(CLAUDE.md/constitution) → 리뷰 게이트 → 이력·감사라는 3계층 방어입니다

리스크	증상	대응
형식주의화	스펙을 형식만 채우고 코드는 따로 놈	스펙 리뷰를 게이트로 강제, 스펙 없는 PR 반려
문서 부담	Spec Kit 도입 후 작은 변경에도 문서 과다	작은 변경은 OpenSpec 델타로, 그린필드만 Spec Kit
AI 과신	AI 리뷰·생성을 무비판 수용	AI 지적도 검증 대상, 사람 판단 게이트 유지
상태 파악 실패	Spec Kit feature 누적으로 지금 동작을 못 봄	living spec이 필요한 도메인은 OpenSpec 선택
하향식 강압	전사 일괄 강제로 반발	자원자 파일럿에서 성공 사례로 자발적 확산
스킬 편차	특정 인원만 잘 씀	CLAUDE.md 온보딩 문서화, 페어링·회고로 전파

예제

도입 의사결정 사례

같은 로드맵도 조직의 규모·유형·성숙도에 따라 도구 선택과 거버넌스 강도가 달라집니다. 그린필드·브라운필드·초심 팀 세 상황을 대비합니다

사례 A · 개발자 15명 신규 SaaS

대부분 그린필드, 규제 대응 필요. Phase 1은 작은 내부 도구로 OpenSpec 파일럿, 익숙해지면 Phase 2에서 신규 제품 라인을 Spec Kit으로. constitution에 보안·규제 원칙을 명문화합니다.

사례 B · 개발자 80명 금융 서비스

10년 된 브라운필드 유지·보수, 감사 요구 강함. OpenSpec 중심 도입으로 델타 이력을 감사 증빙으로 활용하고 changes/archive/ 보존을 거버넌스 규칙으로 확정합니다. 신규 모듈에만 Spec Kit을 선택합니다.

사례 C · 팀 3명 SDD 초심

Phase 1만 집중. OpenSpec 3단계로 습관을 들이고 CLAUDE.md에 최소 규칙(스펙 먼저, 테스트 먼저)만 담습니다. 무리하게 거버넌스를 세우지 않습니다.

실습

우리 팀 도입 워크숍

자가진단, CLAUDE.md 초안, Phase 로드맵을 실제로 써 봅니다. 로드맵은 날짜가 아니라 신호로 넘어가는 것이 핵심입니다

1 **성숙도 자가진단 (10분)**
스펙 없는 PR 반려 여부, 컨벤션 문서 존재, AI 코드 리뷰 도입, 스펙 변경 이력 감사 가능성을 점검해 L1/L2/L3 중 위치와 다음 신호를 정합니다.
진단 4문항 · 다음 단계 신호

2 **우리 팀 CLAUDE.md 초안 (15분)**
기술 스택·아키텍처 원칙 3가지, 스펙 작성 규칙 3가지, 리뷰 게이트 1가지, AI에게 맡기지 않을 범위 2가지를 채우고 온보딩 가능성을 점검합니다.
규율 문서화 · 온보딩 점검

3 **Phase 로드맵 설계 (15분)**
Phase 1 파일럿 팀·프로젝트·도구·기간, Phase 2 확산 범위와 AI 리뷰 시점, Phase 3 거버넌스·감사·지표·혼용 정책을 그립니다. 각 Phase의 완료 신호와 리스크 1개, 대응을 반드시 적습니다.
완료 신호 필수 · 리스크와 대응

개념

정착의 조건 — 규율

SDD의 진짜 자산은 코드가 아니라 스펙입니다

도구를 깐다고 SDD가 도입되는 것이 아닙니다. 규율이 문서로 남고, 리뷰로 강제되고, 이력으로 감사될 때 비로소 조직에 정착합니다.

조직 도입 로드맵과 거버넌스

LO10 핵심 메시지

> 도입은 개인에서 팀, 조직으로 성숙도를 밟아 올라가는 여정입니다. 스펙이 팀 컨벤션·리뷰·이력의 중심에 서야, AI가 코드를 빠르게 써내도 품질과 통제를 잃지 않습니다. 로드맵은 날짜가 아니라 신호로 넘어갑니다.

레퍼런스

조직 도입 방법론(AWS AI-DLC)·두 구현체(OpenSpec, Spec Kit)·AI 코드 리뷰 도구의 1차 출처입니다

방법론 · AWS

AI-DLC

AI 시대 개발 생명주기를 재구성한 오픈소스 방법론입니다. 슬로건은 "AI proposes, human approves"로, 조직 도입 로드맵과 거버넌스의 상위 원리를 제공합니다.

예시: aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/ · re:Invent 2025 DVT214

구현체 · Fission-AI

OpenSpec

단일 living spec과 spec delta로 변경을 관리하는 도구입니다. changes/archive/ 이력이 감사 증빙에 유리해 브라운필드·규제 도입에 권합니다.

예시: github.com/Fission-AI/OpenSpec · npm @fission-ai/openspec v1.5.0(Node 20.19+)

구현체 · GitHub

Spec Kit

Constitution → Specify → Plan → Tasks → Implement 워크플로우를 specify CLI로 실현합니다. feature 폴더 누적이 시점별 근거를 명확히 남겨 그린필드에 유리합니다.

예시: github.com/github/spec-kit · github.github.com/spec-kit/

도구 · 리뷰 게이트

AI 코드 리뷰

2차 AI 리뷰로 스펙 준수·버그·컨벤션을 1차로 걸러냅니다. 복잡 판단은 Opus 4.8($5/$25 per 1M), 일반 리뷰는 Sonnet 4.6($3/$15 per 1M)을 권합니다.

예시: coderabbit(GitHub PR 연동) · 로컬 code-reviewer

체크포인트

스스로 답할 수 있으면 LO10을 통과한 것입니다. 성숙도 진단부터 로드맵 설계까지 여섯 가지를 점검합니다

01 성숙도 진단

개인 → 팀 → 조직 단계를 구분하고, "스펙 없는 PR을 반려하는가" 질문으로 우리 조직 위치를 판단합니다.

02 컨벤션 문서화

CLAUDE.md/constitution에 담는 이유는 AI도 사람도 강제할 수 있기 때문입니다. 머릿속 규칙은 확산되지 않습니다.

03 리뷰 삼중 구조

스펙 리뷰(사람)를 먼저 두고 AI 리뷰로 반복을 걷어낸 뒤, 사람은 판단·설계·보안에 집중합니다.

04 스펙 거버넌스

변경 이력·감사가 왜 필요한지, OpenSpec 아카이브와 Spec Kit feature 폴더가 이를 어떻게 지원하는지 설명합니다.

05 Phase 로드맵

Phase 1 파일럿 → 2 표준화 → 3 확산을 세우고 각 단계의 리스크와 대응을 미리 준비합니다.

06 신호로 넘어간다

로드맵은 날짜가 아니라 완료 신호로 다음 단계로 넘어갑니다. 하향식 강압 대신 성공 사례로 자발적 확산을 유도합니다.

✅ 도구를 까는 것이 아니라 규율을 문서·리뷰·이력으로 남길 때 SDD가 조직에 정착합니다.